



第九章 安全多方计算

姚氏混淆电路

- 两方： 电路生成方+电路求值方
- 电路生成方生成混淆电路以及乱码表
- 电路求值方求值的时候：
 - 两方输入： 生成方将自己输入对应的激活标签直接发给求值方、求值方输入对应的激活标签通过茫然传输协议来获得
 - 逐门计算： 每个门逐一计算
 - 最终结果： 最终输出的激活标签需要发送给生成方来获得

目 录

「05」

知识点五： 行约减

「06」

知识点六： FREE XOR

「07」

知识点七： GMW协议

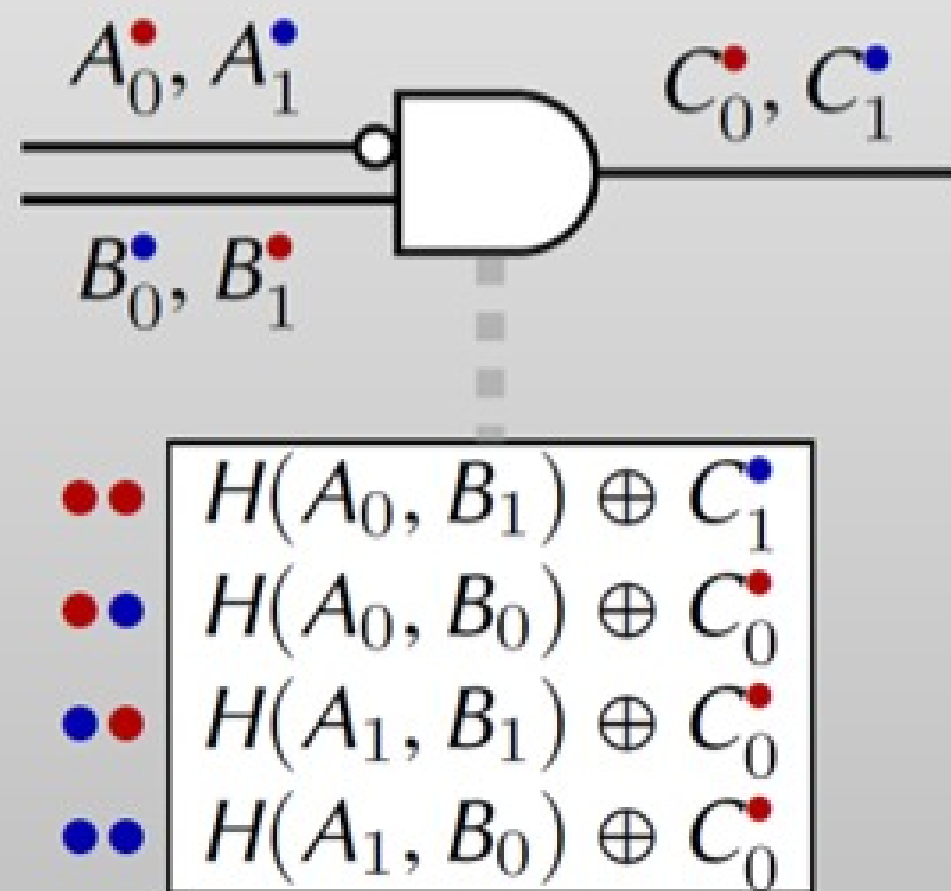
「08」

知识点八： 半树技术

「09」

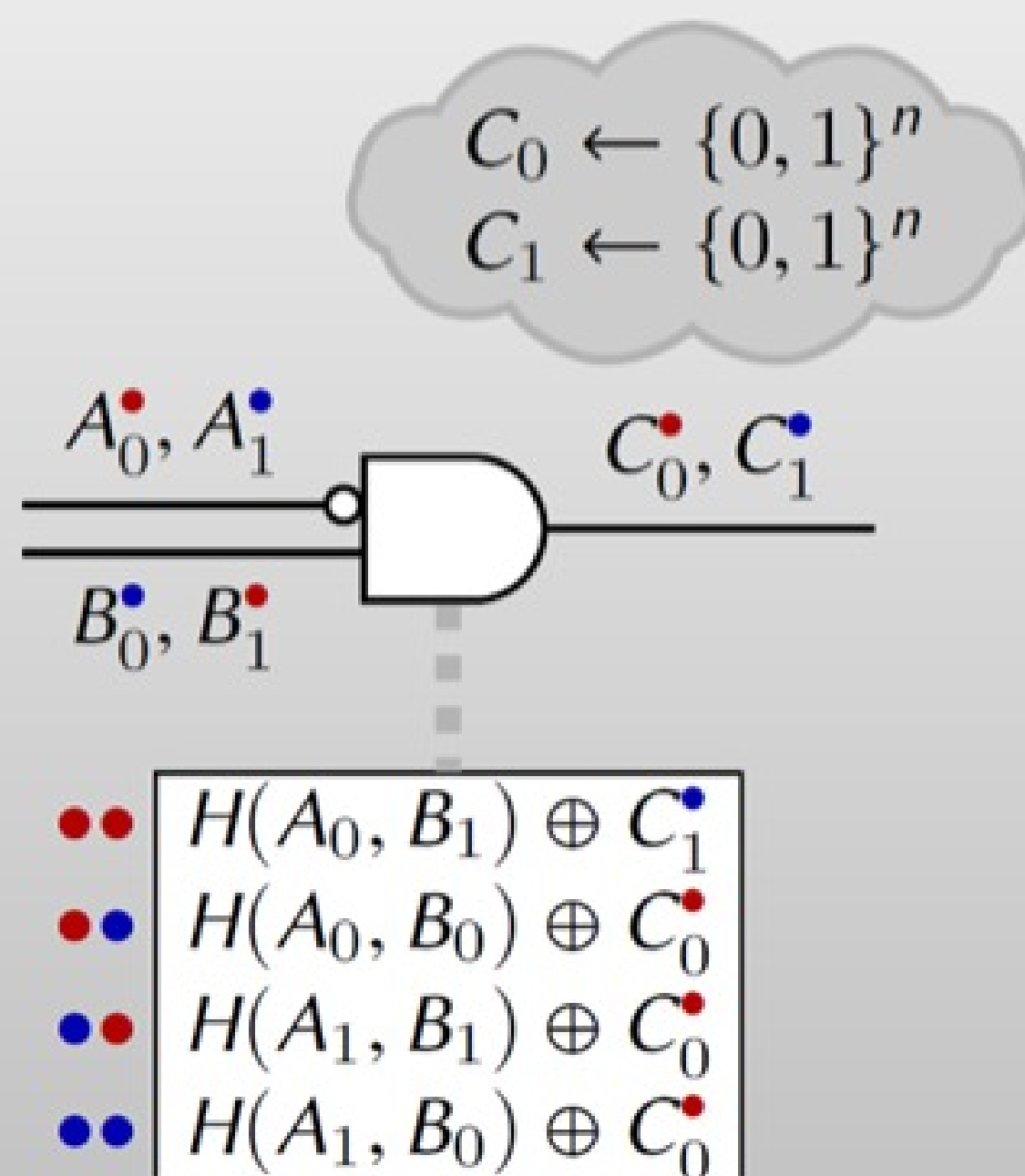
知识点九： 算术电路协议

Garbled Row Reduction [NaorPinkasSumner99]

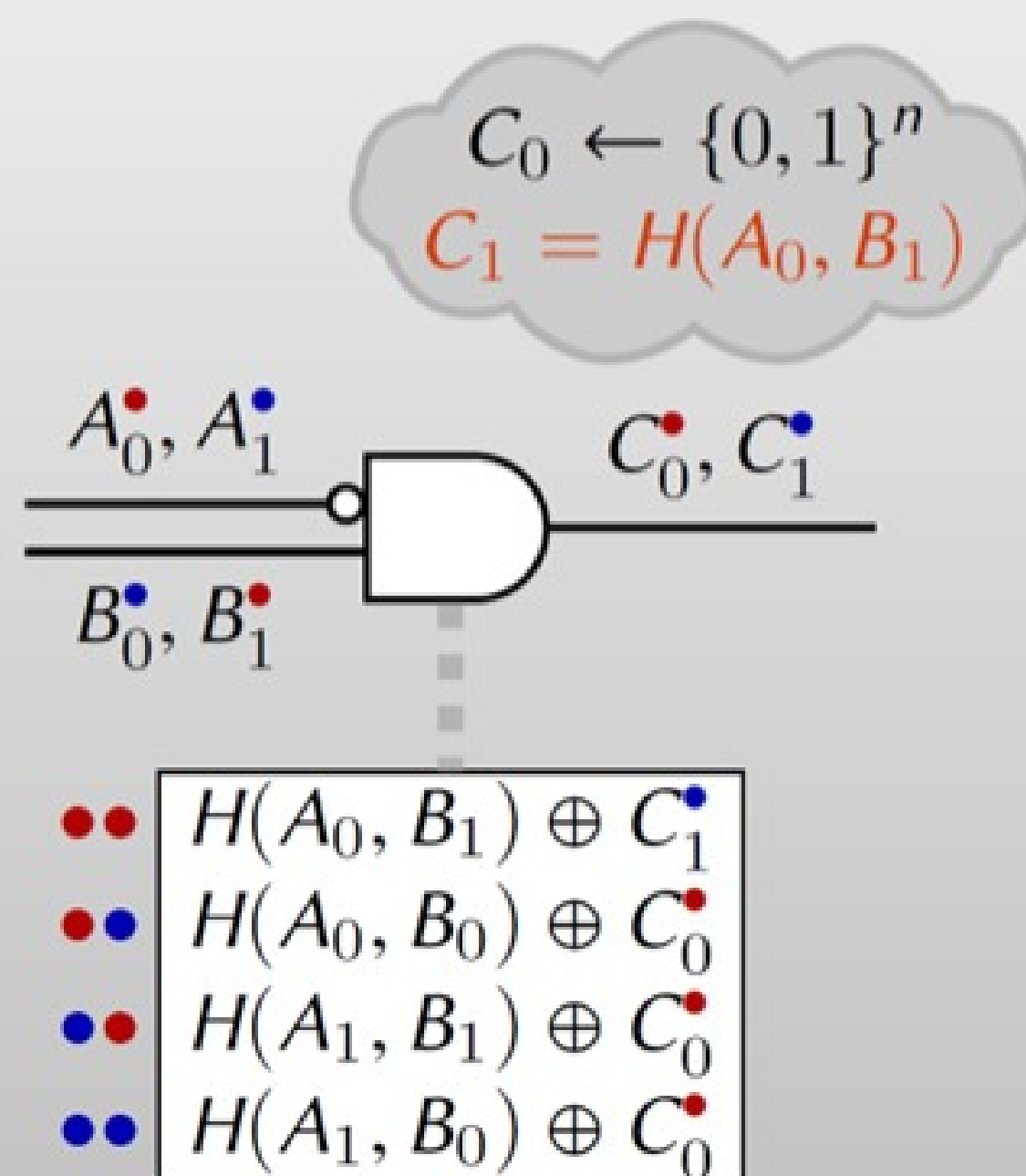


Garbled Row Reduction [NaorPinkasSumner99]

Instead of choosing output wire labels uniformly ...



Garbled Row Reduction [NaorPinkasSumner99]

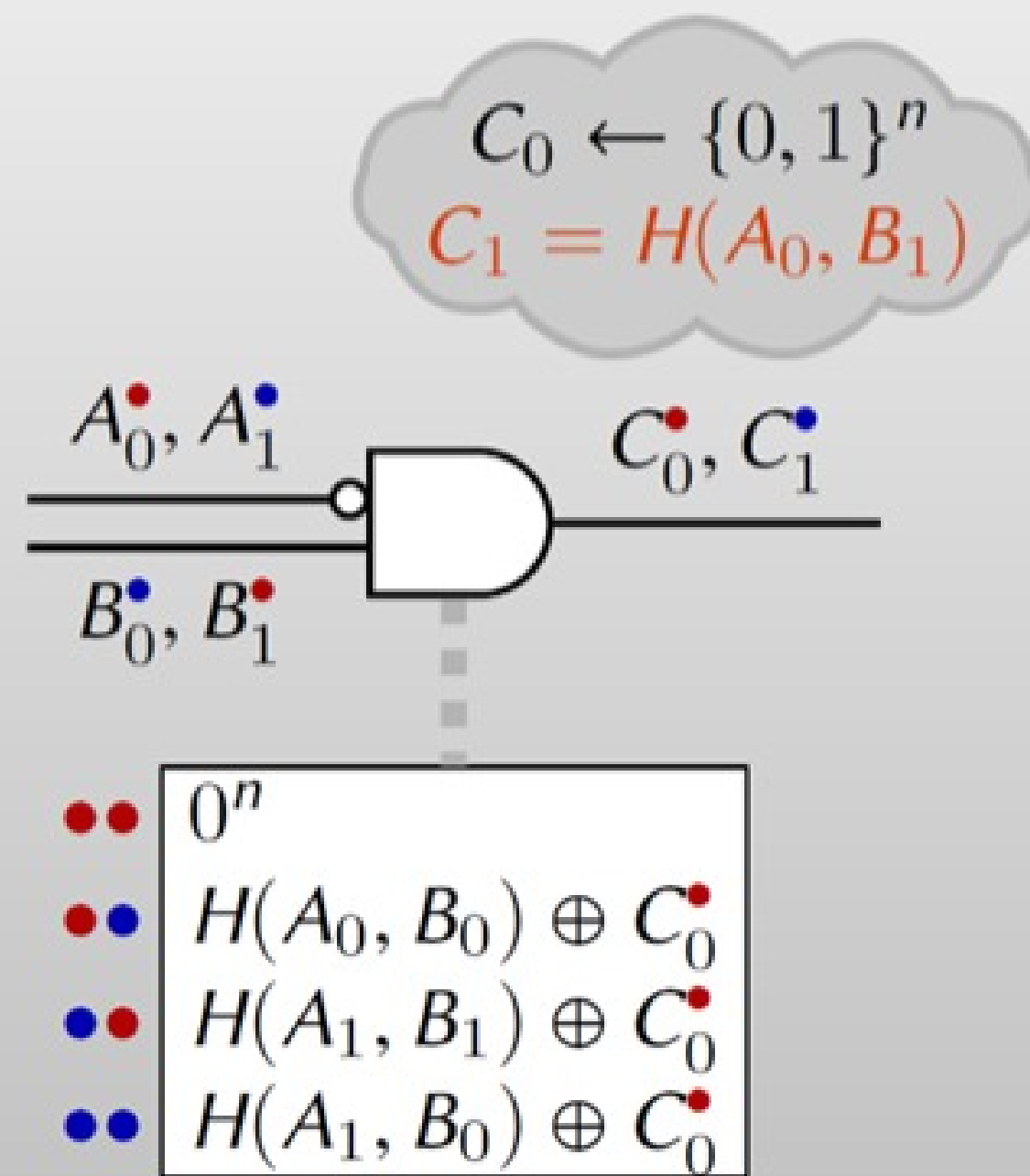


Instead of choosing output wire labels uniformly ...

... choose so that **first ciphertext is 0^n**
(depends on colors & gate function)

第一行对应的输出C不是随机选取
而是由门操作左侧运算 $H(A, B)$ 得到
这样，第一行异或结果为0

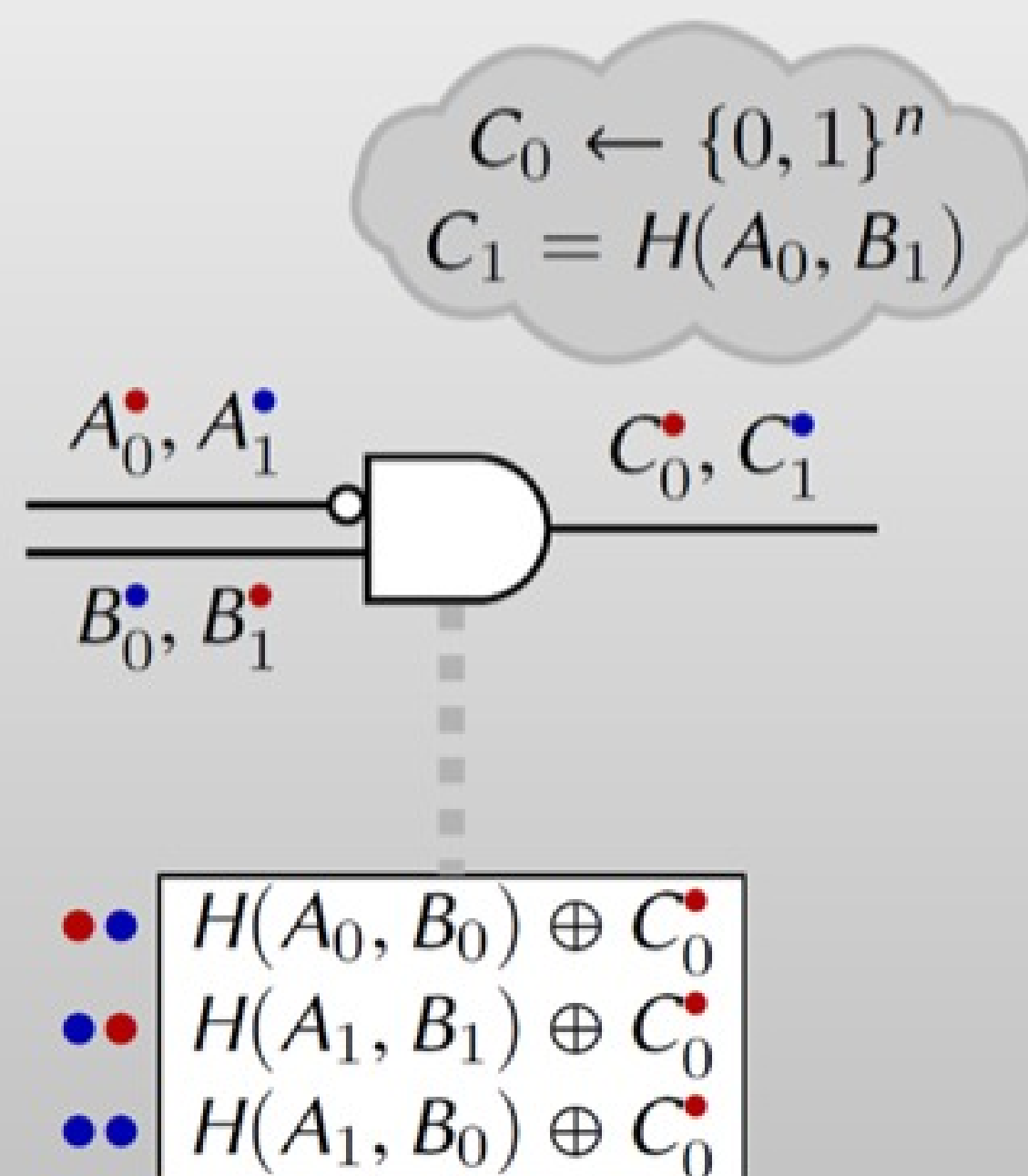
Garbled Row Reduction [NaorPinkasSumner99]



Instead of choosing output wire labels uniformly ...

... choose so that **first ciphertext is 0^n**
(depends on colors & gate function)

Garbled Row Reduction [NaorPinkasSumner99]

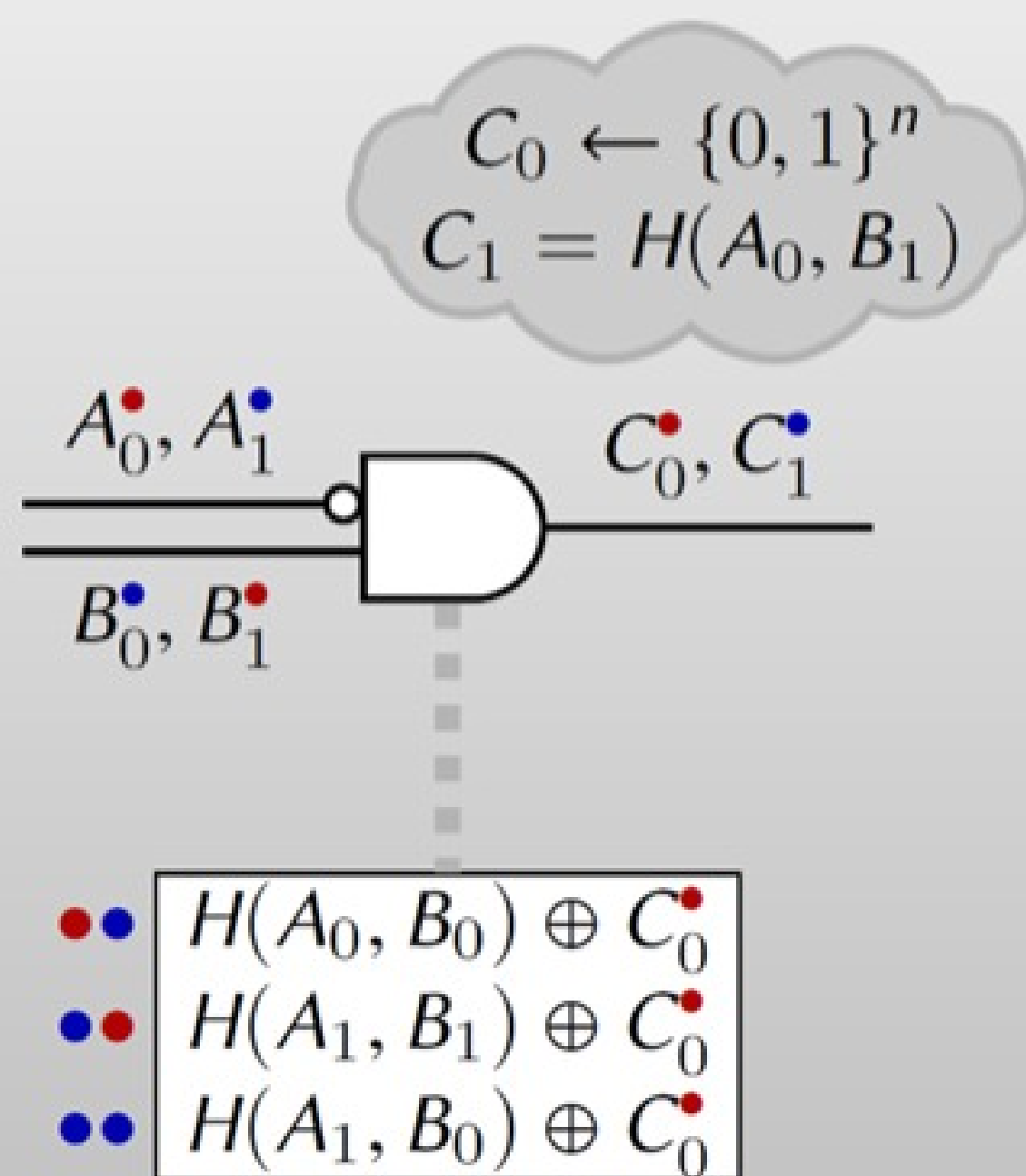


Instead of choosing output wire labels uniformly ...

... choose so that first ciphertext is 0^n
(depends on colors & gate function)

No need to include 1st ciphertext:

Garbled Row Reduction [NaorPinkasSumner99]

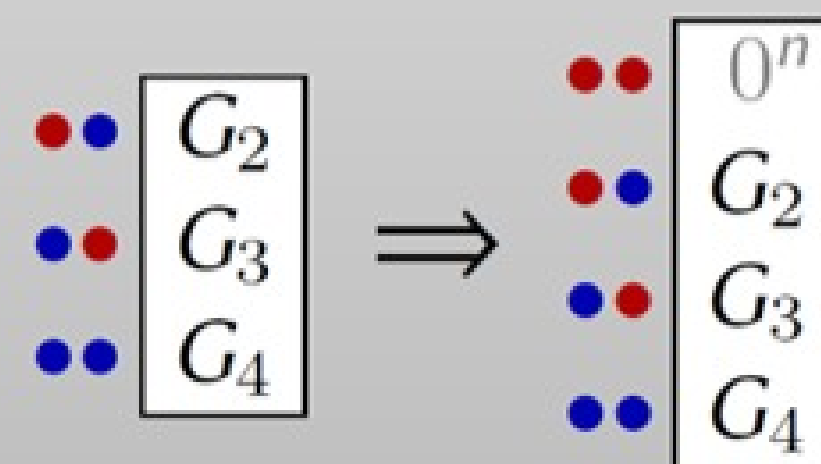


Instead of choosing output wire labels uniformly ...

... choose so that first ciphertext is 0^n
(depends on colors & gate function)

No need to include 1st ciphertext:

- ▶ Evaluator can “reconstruct” missing ciphertext and do the usual thing:



目 录

『05』

知识点五： 行约减

『06』

知识点六： FREE XOR

『07』

知识点七： GMW协议

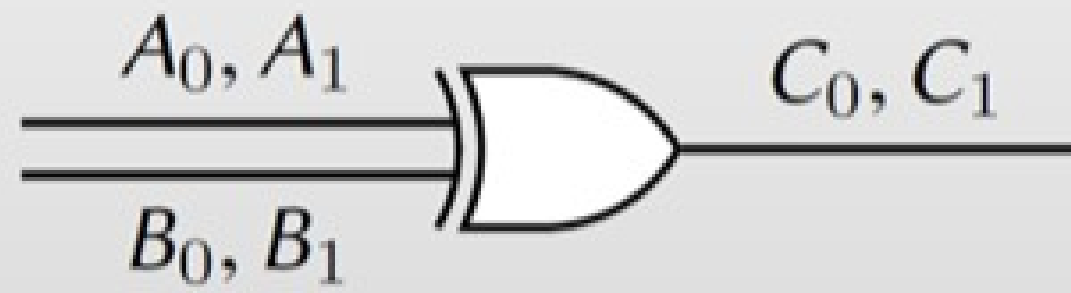
『08』

知识点八： 半树技术

『09』

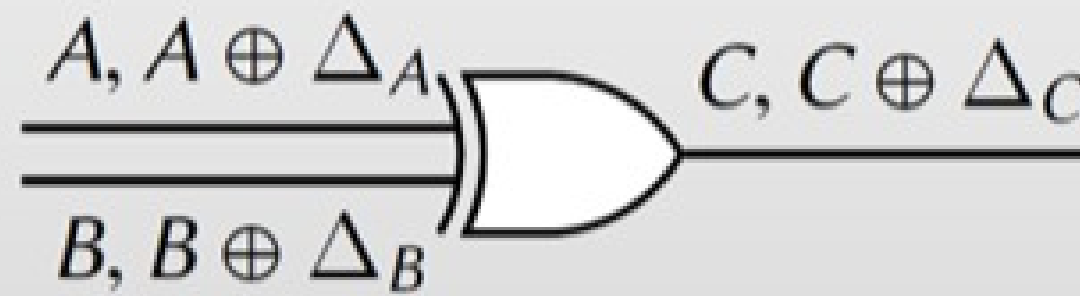
知识点九： 算术电路协议

Free XOR [KolesnikovSchneider08]



- ▶ Define **offset of a wire** $\equiv$ XOR of its two labels

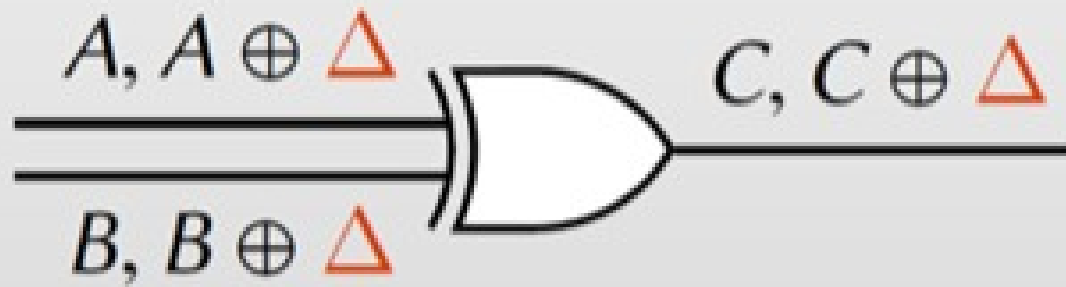
Free XOR [KolesnikovSchneider08]



可以将右侧随机值定义为 $A \oplus$ 某个偏移量
导线值左0、右1

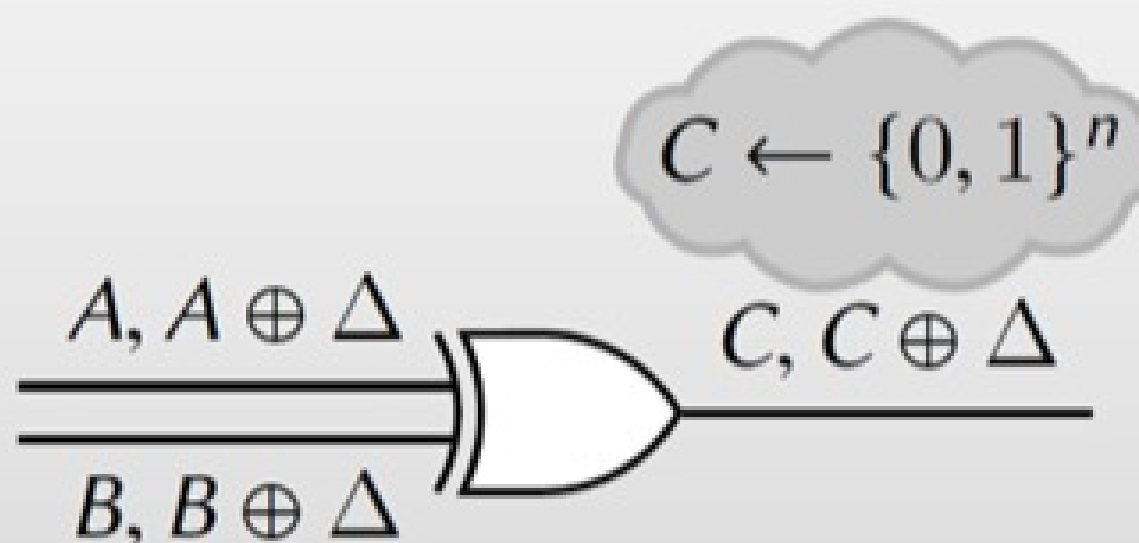
- ▶ Define **offset of a wire** $\equiv$ XOR of its two labels

Free XOR [KolesnikovSchneider08]



- ▶ Define **offset of a wire** $\equiv$ XOR of its two labels
- ▶ Choose all wires in circuit to have same (secret) offset Δ

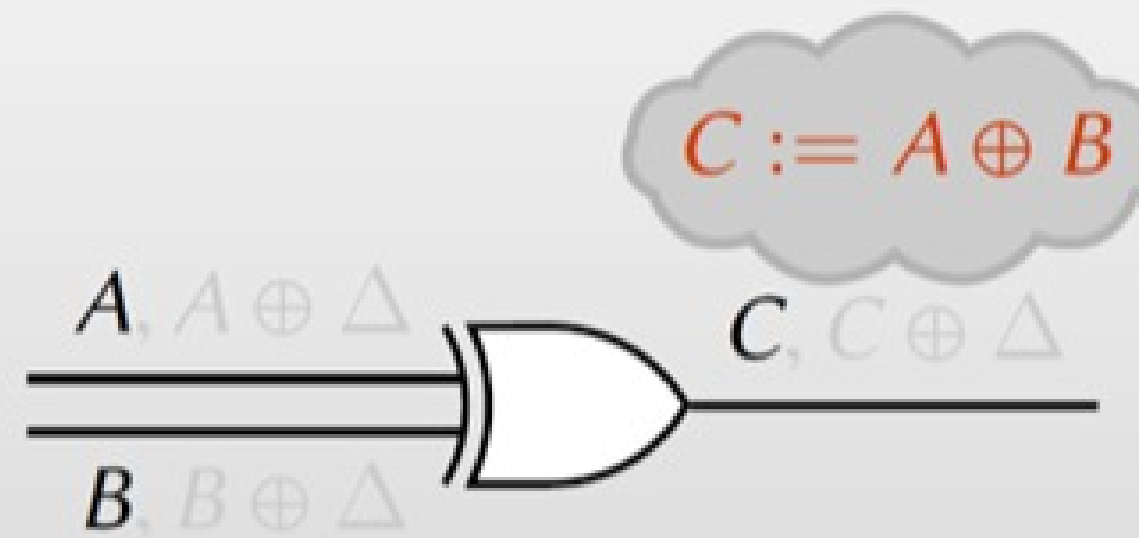
Free XOR [KolesnikovSchneider08]



原先 C 是随机选取的
考虑异或计算的特性，相同为0

- ▶ Define **offset of a wire** $\equiv$ XOR of its two labels
- ▶ Choose all wires in circuit to have same (secret) offset Δ

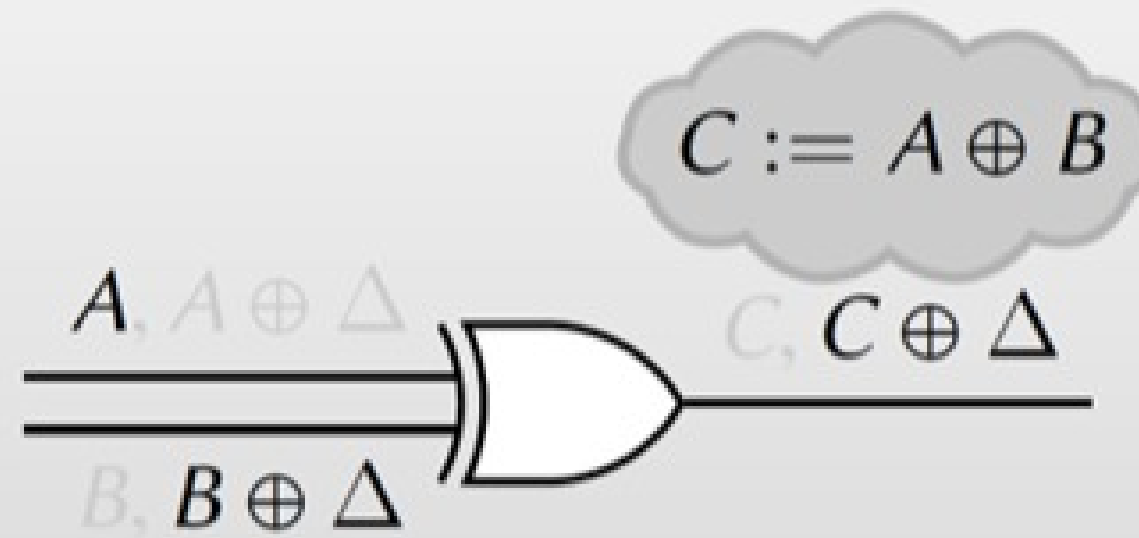
Free XOR [KolesnikovSchneider08]



$$\underbrace{A}_{\text{FALSE}} \oplus \underbrace{B}_{\text{FALSE}} = \underbrace{A \oplus B}_{\text{FALSE}}$$

- ▶ Define **offset of a wire** $\equiv$ XOR of its two labels
- ▶ Choose all wires in circuit to have same (secret) offset Δ
- ▶ Choose FALSE output = FALSE input $\oplus$ FALSE input

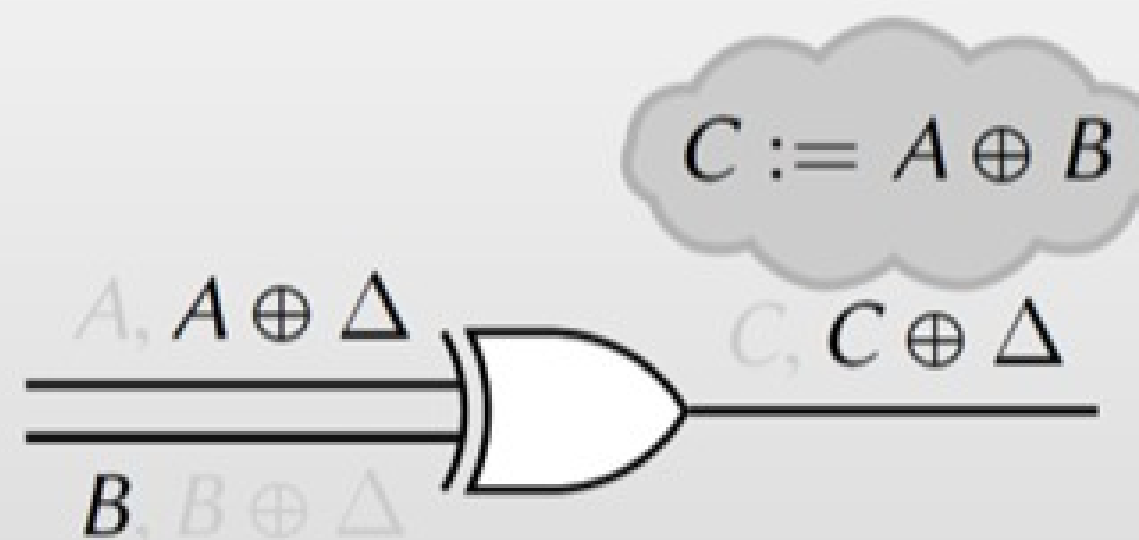
Free XOR [KolesnikovSchneider08]



$$\underbrace{A}_{\text{FALSE}} \oplus \underbrace{B \oplus \Delta}_{\text{TRUE}} = \underbrace{A \oplus B \oplus \Delta}_{\text{TRUE}}$$

- ▶ Define **offset of a wire** $\equiv$ XOR of its two labels
- ▶ Choose all wires in circuit to have same (secret) offset Δ
- ▶ Choose FALSE output = FALSE input $\oplus$ FALSE input
- ▶ Evaluate by xoring input wire labels (no crypto)

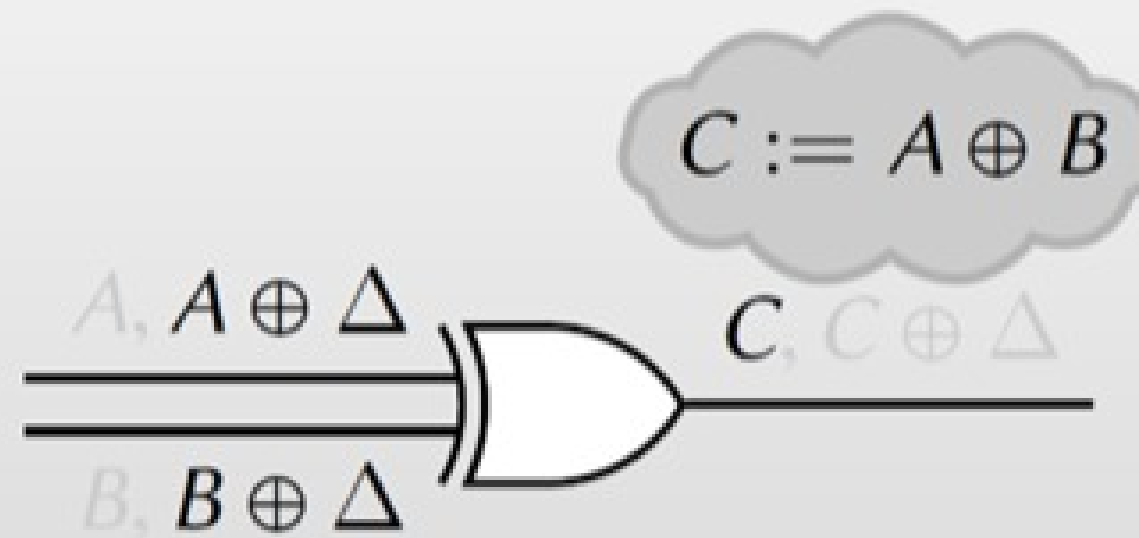
Free XOR [KolesnikovSchneider08]



$$\underbrace{A \oplus \Delta}_{\text{TRUE}} \oplus \underbrace{B}_{\text{FALSE}} = \underbrace{A \oplus B \oplus \Delta}_{\text{TRUE}}$$

- ▶ Define **offset of a wire** $\equiv$ XOR of its two labels
- ▶ Choose all wires in circuit to have same (secret) offset Δ
- ▶ Choose FALSE output = FALSE input $\oplus$ FALSE input
- ▶ Evaluate by XORing input wire labels (no crypto)

Free XOR [KolesnikovSchneider08]



$$\underbrace{A \oplus \Delta}_{\text{TRUE}} \oplus \underbrace{B \oplus \Delta}_{\text{TRUE}} = \underbrace{A \oplus B}_{\text{FALSE}}$$

- ▶ Define **offset of a wire** $\equiv$ XOR of its two labels
- ▶ Choose all wires in circuit to have same (secret) offset Δ
- ▶ Choose FALSE output = FALSE input $\oplus$ FALSE input
- ▶ Evaluate by xoring input wire labels (no crypto)

□ **FreeXOR**要求生成乱码电路所有导线标签时，都要使用相同的偏移量 Δ
可以基于选定的偏移量，逐门产生每条导线的标签

□ 姚氏乱码电路要为XOR门生成完整的乱码表，求值过程也需要解密乱码表

□ 使用**FreeXOR**技术后，**XOR**门的求值过程不需要乱码表

目 录

『05』

知识点五： 行约减

『06』

知识点六： FREE XOR

『07』

知识点七： GMW协议

『08』

知识点八： 半树技术

『09』

知识点九： 算术电路协议

1.直观思想

- 在姚氏乱码电路中，实现激活导线值秘密分享的方法是让一个参与方（电路生成方）持有两个可能的导线标签 w_i^0 和 w_i^1 ，另一个参与方（电路求值方）持有激活标签 w_i^b ，该激活标签是通过茫然传输协议（公钥密码机制）来完成的。
- 2004年，Goldreich等基于秘密共享提出GMW协议，实现导线值秘密分享的方法更加直接：让各个参与方持有激活导线值的加法秘密份额，在计算的时候尽可能避免调用茫然传输协议（公钥密码机制）来提升运算效率。

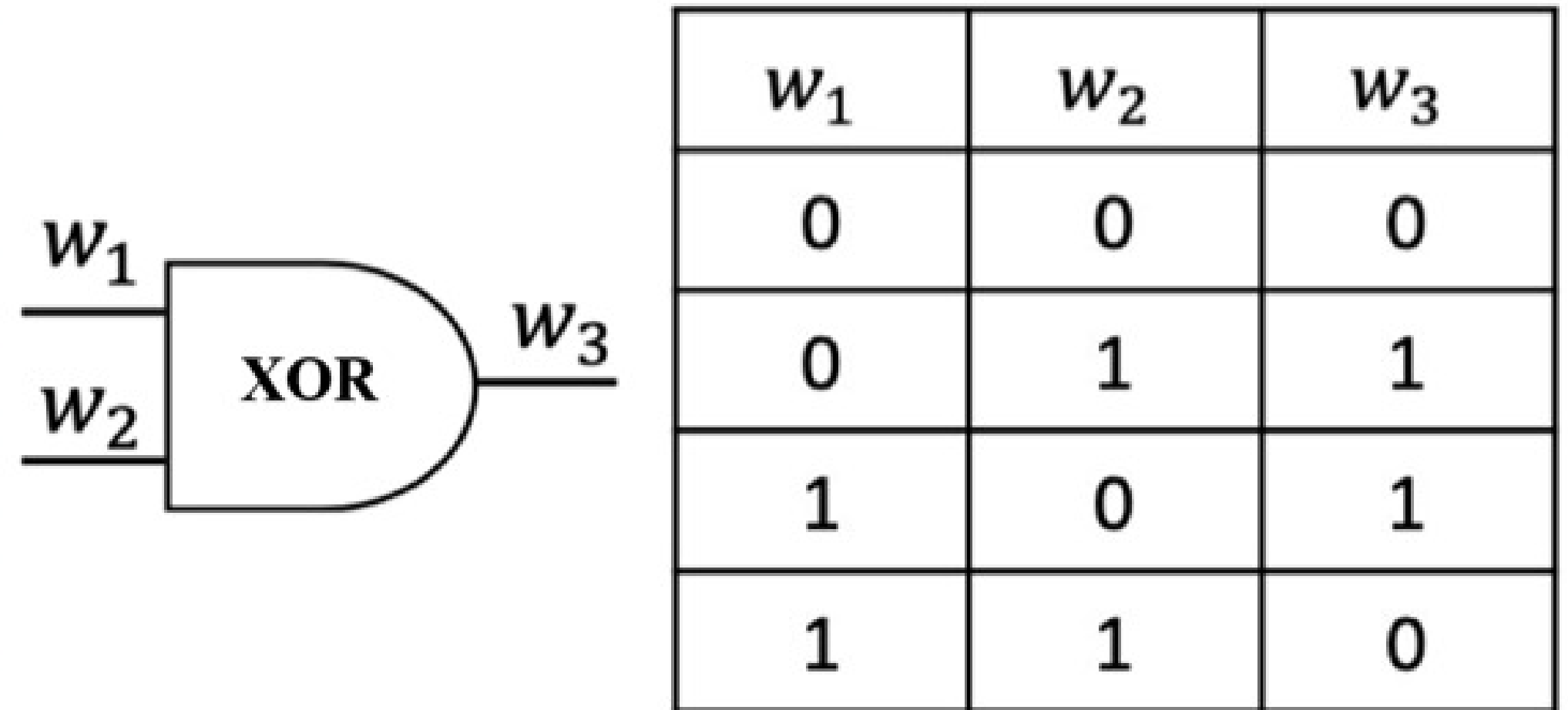
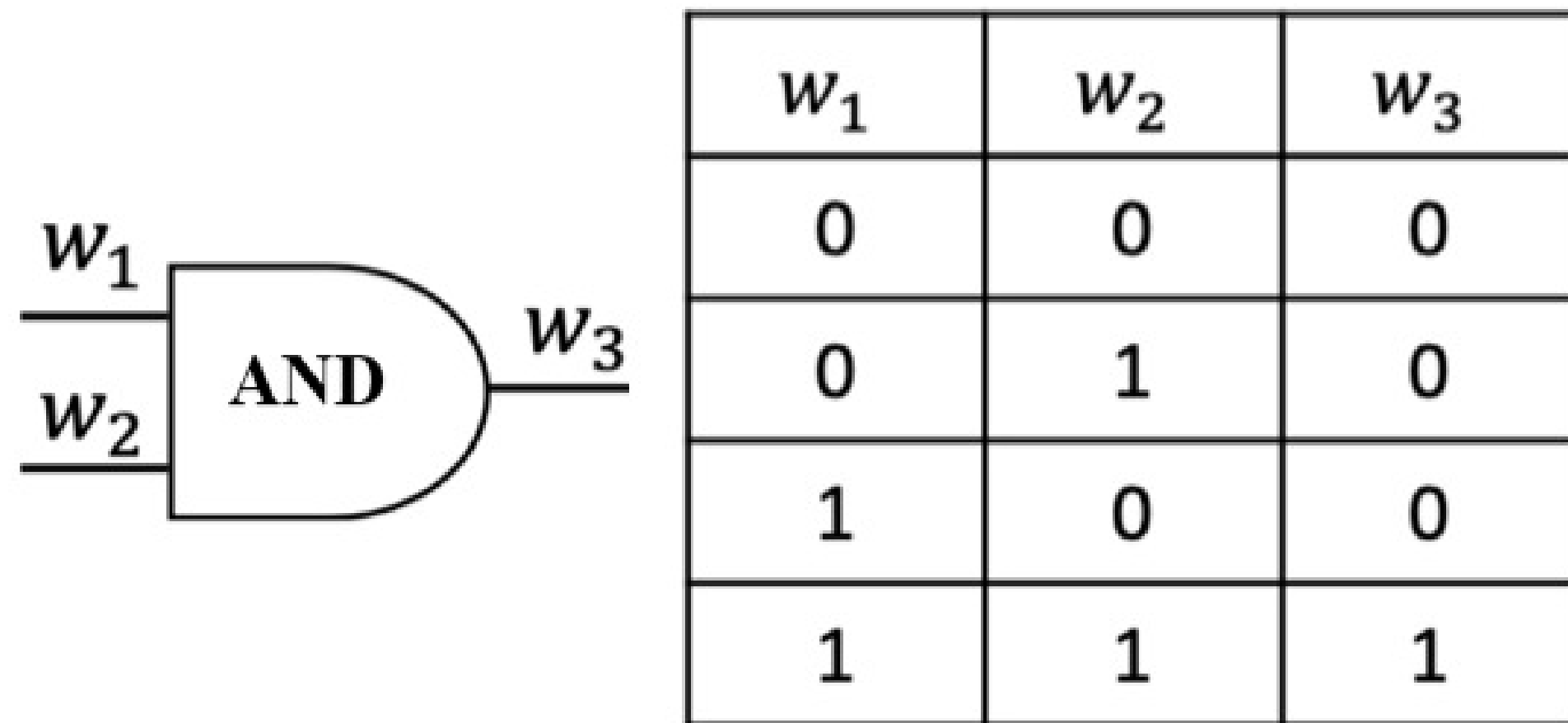
1.直观思想

- GMW 协议同时支持布尔电路和算术电路。构造协议的目的是完成多方安全运算某一约定好的函数的功能。
- 与姚氏混淆电路相同：GMW主要的实现方式在于将所需计算的函数转换为函数所对应的电路，而电路的基本单元可以归约为单个的门电路。协议设计了针对每种基本门电路的密文运算方法。
- 姚氏混淆电路不同：在整体协议构造中，在计算参与方协商好所需计算的函数所对应的电路后，参与方将参与计算的数据秘密共享至所有参与者，再以密文形式进行逐个门电路的运算。

2.具体构造

2.1 准备知识

GMW协议的目标函数由XOR（异或）门和AND（与）门、NOT（非）门组成。NOT门的输出值是输入值的反，输入为1则输出为0，反之输入为0则输出为1。AND门和XOR门的真值表如下图所示：



2.具体构造

2.2 两方协议流程

针对**GMW**协议的2方情况：

- 设协议参与方为 P_1 和 P_2 ，参与方 P_1 的输入为 a ，参与方 P_2 的输入为 b 。
- **GMW**协议执行需要各数据持有方将需要加入计算的输入进行加法秘密分享，继而在秘密分享后的数据上进行门电路的运算。

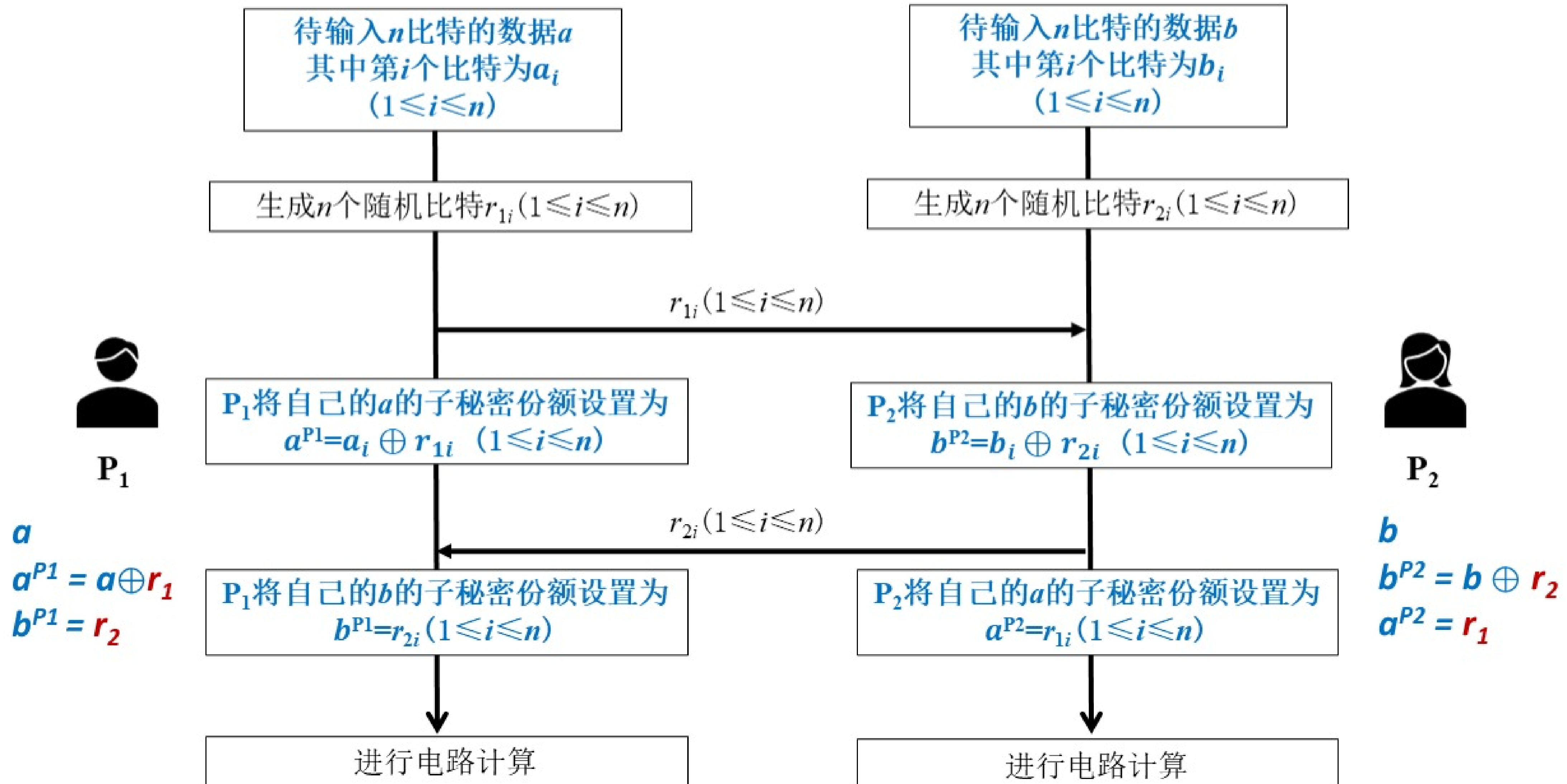
2.具体构造

2.2 两方协议流程

○ 加法秘密分享

- 对于 $a \in \{0,1\}^n$ 中的每一个输入比特 $a_i \in \{0, 1\}$ ， P_1 生成一个随机比特 $r_i \in_R \{0, 1\}$ ，并将所有的 r_i 发送给 P_2 。接下来， P_1 将自己的秘密份额设置为 $a_i \oplus r_i$ ，相当于 P_1 和 P_2 ，对每一个 a_i 进行了加法秘密分享。
- P_2 也为其每一个输入比特 b_i 生成随机比特掩码，并将掩码发送给 P_1 ，以类似的方法实现对 b_i 的加法秘密分享。
- 两方输入数据的加法秘密分享流程图如下页所示：

2.具体构造



2.具体构造

○ 电路计算

接着 P_1 和 P_2 逐一对布尔电路中的每个门求值，直至完成整个电路的计算。

由于GMW 协议的目标函数由XOR门和AND门、NOT门组成。考虑三种门电路在加法秘密分享下的计算方式如下：

□NOT门

NOT门是单输入的， P_1 和 P_2 间无须进行交互，因此直接对输入求反即可。

以经过秘密分享的数据 a 为例： P_1 和 P_2 分别持有 a 的子秘密份额（记为 a^{P1} 和 a^{P2} ），则为了计算以 a 为输入的NOT门输出结果 $\neg a$ 的加法秘密分享值，由 P_1 直接本地对 a^{P1} 取反， P_2 直接本地对 a^{P2} 取反，所得的新份额即为所求 $\neg a$ 的加法秘密分享份额。

□NOT门



P_1

a

$$a^{P1} = a \oplus r_1$$

$$b^{P1} = r_2$$



P_2

b

$$b^{P2} = b \oplus r_2$$

$$a^{P2} = r_1$$

以经过秘密分享的数据 a 为例，
如何求 $\neg a$ 的加法秘密分享值？

- 由 P_1 直接本地对 a^{P1} 取反， P_2 直接本地对 a^{P2} 取反
- 所得的新份额即为所求 $\neg a$ 的加法秘密分享份额

$$\neg a^{P1} = \neg a \oplus \neg r_1$$

$$\neg a^{P2} = \neg r_1$$

$$\text{验证} \implies \neg a^{P1} \oplus \neg a^{P2} == \neg a$$

2.具体构造

□ XOR门

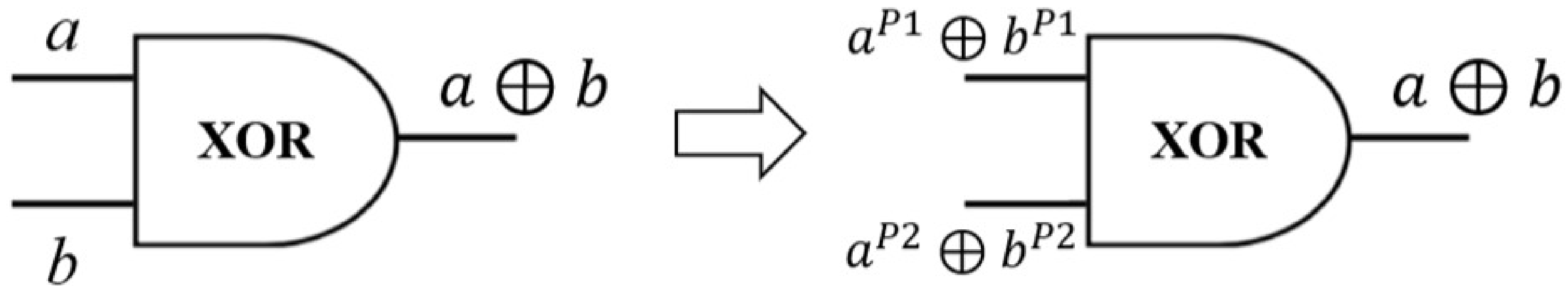
对于XOR门，假设异或门的输入分别为 a 和 b ， P_1 掌握了 a 和 b 的子秘密 a^{P1} 和 b^{P1} ， P_2 掌握了 a 和 b 的子秘密 a^{P2} 和 b^{P2} ，因为：

$$\begin{aligned} & (a^{P1} \oplus b^{P1}) \oplus (a^{P2} \oplus b^{P2}) \\ &= (a^{P1} \oplus a^{P2}) \oplus (b^{P1} \oplus b^{P2}) \\ &= a \oplus b \end{aligned}$$

因此， P_1 和 P_2 在本地对其所掌握的子秘密进行计算可以得到输出的秘密共享份额，将来计算输出的时候，不需要乱码表，只需要互相共享各自的秘密份额就可以了。

2.具体构造

□ XOR门



P_1 和 P_2 在本地对其所掌握的子秘密进行计算可以得到输出的秘密共享份额，将来计算输出的时候，不需要乱码表，只需要互相共享各自的秘密份额就可以了。

2.具体构造

□ AND门

对于AND门，依然假设AND门的输入分别为 a 和 b ， P_1 掌握了 a 和 b 的子秘密 a^{P1} 和 b^{P1} ， P_2 掌握了 a 和 b 的子秘密 a^{P2} 和 b^{P2} 。

P_1 和 P_2 无法只在本地完成AND门的计算，原因如下式：

$$\begin{aligned} a \wedge b &= (a^{P1} \oplus a^{P2}) \wedge (b^{P1} \oplus b^{P2}) \\ &= (a^{P1} \wedge b^{P1}) \oplus (a^{P1} \wedge b^{P2}) \oplus (a^{P2} \wedge b^{P1}) \oplus (a^{P2} \wedge b^{P2}) \end{aligned}$$

因为 P_1 手中有 a^{P1} 和 b^{P1} ，因此 P_1 可以本地计算 $(a^{P1} \wedge b^{P1})$ 。 P_2 手中有 a^{P2} 和 b^{P2} ，因此 P_2 可以本地计算 $(a^{P2} \wedge b^{P2})$ 。但是，剩下的 $(a^{P1} \wedge b^{P2})$ 和 $(a^{P2} \wedge b^{P1})$ 二者不能在本地计算。也就是说， P_1 和 P_2 在本地对其所掌握的子秘密进行计算是无法得到AND操作的输出的秘密共享份额，我们必须协同交互来完成该操作。

2.具体构造

因为 $a \wedge b = (a^{P1} \oplus a^{P2}) \wedge (b^{P1} \oplus b^{P2})$ ，如前所述，可以如下完成AND门计算：

- P_2 掌握的秘密份额 a^{P2} 和 b^{P2} 都是单比特值，所以 (a^{P2}, b^{P2}) 的罗列后存在4种情况为： $(0,0), (0,1), (1,0), (1,1)$ ；
- P_1 手中又有 a^{P1} 和 b^{P1} ，因此 P_1 可以直接计算出四种情况下的 $a \wedge b$ 的值，假设分别为 $S(0,0), S(0,1), S(1,0), S(1,1)$ ；
- 进而， P_2 将其所掌握的秘密份额 a^{P2}, b^{P2} 作为输入，通过执行四选一不经意传输协议来得到对应的 $S(a^{P2}, b^{P2})$ ；
- 考虑到安全计算的需要， $S(a^{P2}, b^{P2})$ 也需要保密， P_1 将通过产生一个随机数来生成该数据的秘密份额。

2.具体构造

首先， P_1 首先以 P_2 拥有的秘密份额 a^{P_2} 和 b^{P_2} 的取值，计算得到下表：

a^{P_2}	b^{P_2}	$a \wedge b$
0	0	$S(0,0)$
0	1	$S(0,1)$
1	0	$S(1,0)$
1	1	$S(1,1)$

2.具体构造

其次， P_1 选择一个随机掩码比特 $r \in_R \{0,1\}$ ，并准备一张OT秘密输入表：

$$T_G = \begin{Bmatrix} r \oplus S(0,0) \\ r \oplus S(0,1) \\ r \oplus S(1,0) \\ r \oplus S(1,1) \end{Bmatrix}$$

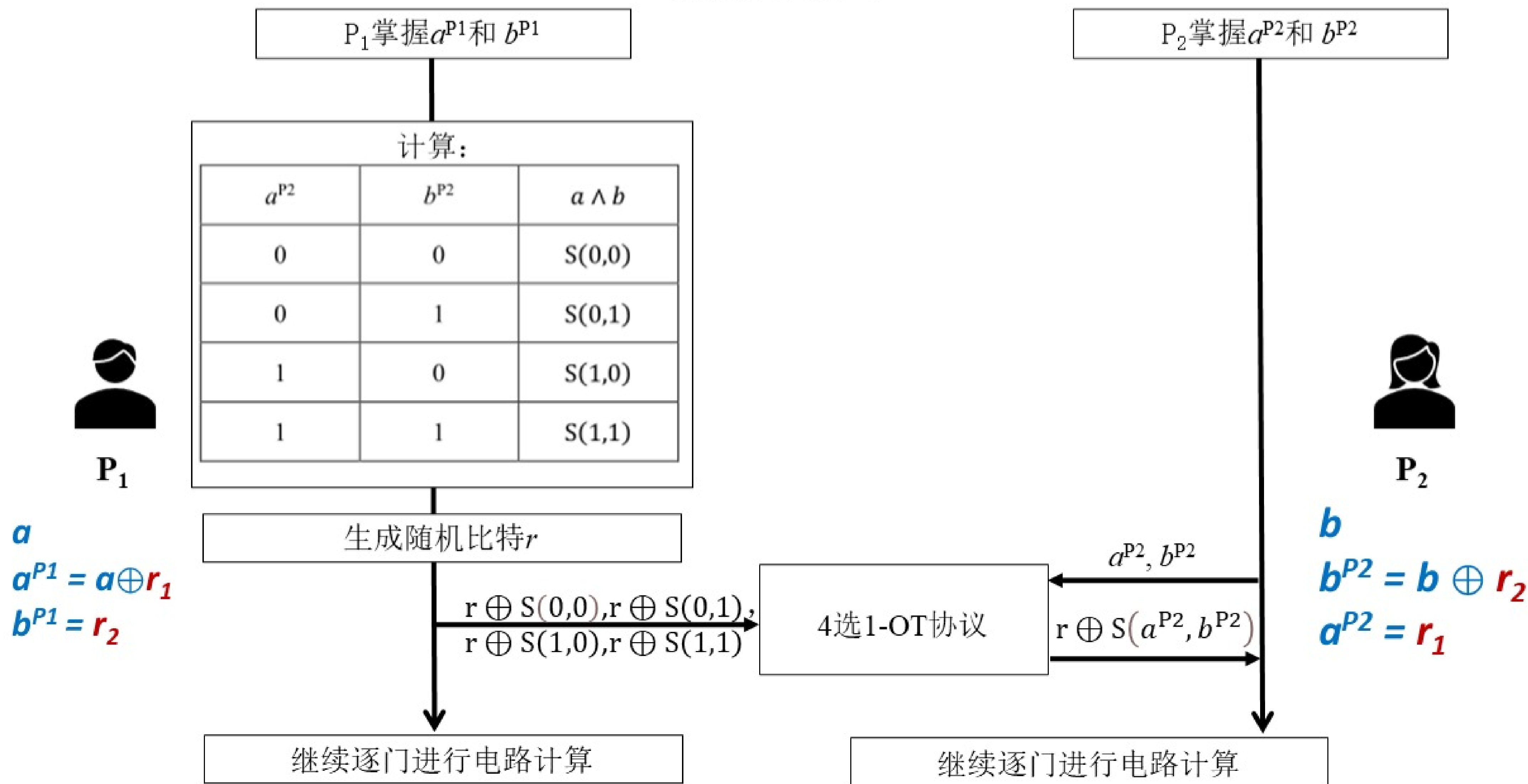
然后， P_2 将其所掌握的 a^{P_2} ， b^{P_2} 作为四选一不经意传输协议的输入，双方执行该OT协议，从而 P_2 得到对应的 $r \oplus S(a^{P_2}, b^{P_2})$ 。

最终结果是： P_1 将 r 作为该AND门的输出的子秘密， P_2 将 $r \oplus S(a^{P_2}, b^{P_2})$ 作为该AND门的输出的子秘密。

2.具体构造

- 由于 P_1 和 P_2 双方间执行的是不经意传输协议， P_1 不知道 P_2 在4个输入中选择了哪个，保证了 a^{P_2} ， b^{P_2} 依旧对 P_1 保密。又由于 P_1 发送的是 $r \oplus S(a^{P_2}, b^{P_2})$ ， P_2 不知道随机比特 r 的值，也就不知道 $S(a^{P_2}, b^{P_2})$ 到底是1还是0，只有在最后双方都公布各自手中的计算结果，双方才能协同计算出正确的结果。
- 这里 P_2 得到的子秘密异或上 P_1 得到的子秘密即为 $a \wedge b$ 的值。
- 本部分所述两方计算AND门电路的运算整体过程图示如后页：

2.具体构造



2.具体构造

2.3 扩展到多方

将GMW协议扩展到多方情况下：即参与者为 $P_1, P_2, \dots, P_n$ 。

○ 加法秘密分享

假设输入为 m 个比特，每个参与者 P_i 产生 m 组，每组 $(n-1)$ 个随机比特，分别将每组的 $(n-1)$ 个随机比特发送给除了自己之外的 $(n-1)$ 个参加者。

假设输入数据为1个比特，那么每个参与者 P_i 向参与者 P_j 发送 r_{ij} ，其中 $1 \leq j \leq n$ 且 $j \neq i$ 。在所有参与者发送完成后，参与者 P_i 掌握了 $r_{1i}, r_{2i}, \dots, r_{ni}$ 。假设 P_i 的输入为 a ，那么 P_i 将 $a \oplus r_{1i} \oplus r_{2i} \oplus \dots \oplus r_{ni}$ 作为 a 的子秘密 a^{P_i} 。

对应的收到 P_i 发送的随机比特 r_{ij} 的参与者 P_j ，将 r_{ij} 当作秘密 a 的子秘密 a^{P_j} 。因此 $a = a^{P_1} \oplus a^{P_2} \oplus \dots \oplus a^{P_n}$ 。

2.具体构造

○ 电路计算

对于**XOR**门以及**NOT**门，和两方情况下相同，每个参与者直接在本地进行计算即可。

对于AND门，依然设与门的输入为 a 和 b ，参与者 P_i 手中掌握了 a 和 b 的子秘密 a^{P_i} 和 b^{P_i} ，因为 $a=a^{P_1}\oplus a^{P_2}\oplus\cdots\oplus a^{P_n}$ ， $b=b^{P_1}\oplus b^{P_2}\oplus\cdots\oplus b^{P_n}$ ，可得：

$$\begin{aligned}a \wedge b &= (a^{P_1} \oplus a^{P_2} \oplus \cdots \oplus a^{P_n}) \wedge (b^{P_1} \oplus b^{P_2} \oplus \cdots \oplus b^{P_n}) \\&= \left(\bigoplus_{i=1}^n a^{P_i} \wedge b^{P_i}\right) \oplus \left(\bigoplus_{i \neq j} a^{P_i} \wedge b^{P_j}\right)\end{aligned}$$

这其中，对于 P_i 来说，每个 $a^{P_i} \wedge b^{P_i}$ 都可由 P_i 在本地进行计算。而对于 $a^{P_i} \wedge b^{P_j}, i \neq j$ ，可由参与者 P_i 和 P_j 通过前述的两方GMW协议共同计算出 $a^{P_i} \wedge b^{P_j}$ 的两个子秘密。当电路计算完成，每个参与者公布自己所掌握的子秘密，再将所有的子秘密进行异或，即可得到最后的计算结果。

目 录

『05』

知识点五： 行约减

『06』

知识点六： FREE XOR

『07』

知识点七： GMW协议

『08』

知识点八： 半树技术

『09』

知识点九： 算术电路协议

汇报目录

BGW协议

1.直观思想

由Ben-Or等人在1988年提出的BGW_a协议是首批支持多个参与方计算的 MPC协议之一。通常将Ben-Or等人提出的协议称为“BGW”协议。我们这里直接介绍n个参与方的BGW协议，这个协议描述起来会相对简单一些。

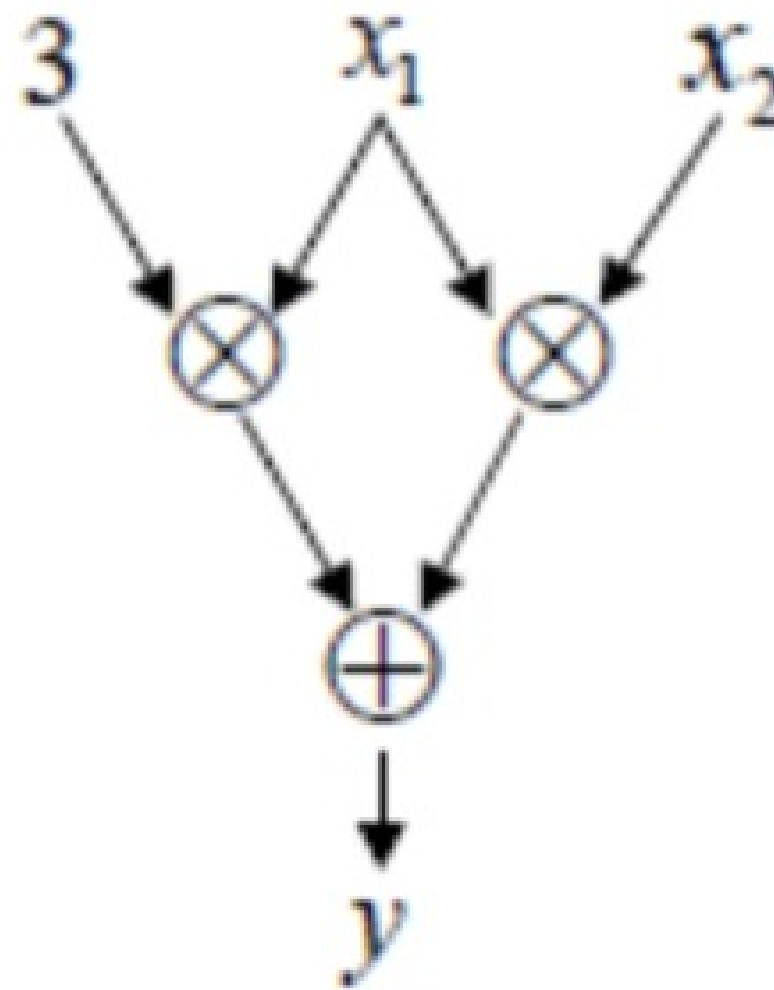
BGW协议可以用于对域F上包含加法、乘法、常数乘法门的算术电路求值。
此协议强依赖于Shamir秘密分享方案（Shamir, 1979），巧妙地利用了Shamir秘密分享方案的同态特性——对各个秘密份额进行适当的处理，就可以在秘密值上实现安全计算。

1.直观思想

算术电路和布尔电路。在工程和计算机科学中，函数被经典地表示为布尔电路，布尔电路基本上有逻辑门（与门、或门、非门等）和连接它们的导线组成。如前面姚氏混淆电路和GMW协议里描述的一样，一根导线是1个比特位，逻辑门是基本单元，它可以处理两个比特位的与或非等基本运算。对于比特串上的与或运算，需要通过定义多个逐位的基本逻辑门进行并联。

算术电路函数的更紧凑表示是算术电路，不像布尔电路的导线选择空间是 $\mathbb{Z}_2$ ，算术电路导线的值从 $\mathbb{Z}_{m \geq 2}$ 中选择。门操作为模加法或模乘法，下图表示了一个简单的算术电路。

1.直观思想



可以将任何布尔电路表示为 $\mathbb{Z}_2$ 上的算术电路，但是，如果 $\mathbb{Z}_m$ 的模 m 足够大，那么得到的函数的算术电路表示可能比其布尔电路表示的大小要小得多，因为对于每个整数的加法或乘法来说，一次操作就足够了。

1.直观思想

BGW协议直观思想。在GMW虽然也使用了秘密分享，但采用的是位异或的分享方法，在算术电路实现运算开销比较大。**BGW**协议则强依赖于**Shamir**秘密分享方案，巧妙地利用了**Shamir**秘密分享方案的同态特性—对各个秘密份额进行适当的处理，就可以在秘密值上实现安全计算。

给定 $v \in F$ ，我们令 $[v]$ 表示各个参与方持有 v 的Shamir秘密份额。具体来说，某一参与方选择一个阶最高为 t 的随机多项式 p ，并令 $p(0)=v$ 。每个参与方 P_i 把 $p(i)$ 作为 v 的秘密份额。我们称 t 为秘密分享的阈值，即任意 t 个秘密份额都不会泄漏与 v 相关的任何信息。

BGW协议的固定范式为：对于算术电路的每一条导线 w ，各个参与方都持有导线值 v_w 所对应的秘密份额 $[v_w]$ ；每个电路门进行计算的时候，通过拥有的秘密份额（在乘法操作需要交互）完成输出值的秘密份额的计算。

2.具体构造

○ 秘密共享

假设BGW协议的参与者一共有 n 方： $P_1, P_2, \dots, P_n$ ，假设参与者 P_i 需要输入秘密 a ，则参与者 P_i 首先利用Shamir(t, n)门限秘密共享机制将秘密 a 共享给其他所有参与者，阈值 t 的选择根据具体使用情景下的安全性要求决定。

当所有参与者的输入都通过Shamir(t, n)门限秘密共享机制分享后，每个参与者都掌握了协议输入的子秘密。假设一个门的输入分别为 a 和 b ，秘密 a 和 b 已经分别由秘密分配函数

$$f_a(x) = a_{t-1}x^{t-1} + \dots + a_1x^1 + a$$

$$f_b(x) = b_{t-1}x^{t-1} + \dots + b_1x^1 + b$$

分配完成， $f_a(0)=a$ ， $f_b(0)=b$ ，参与者 P_i 掌握 a 和 b 的子秘密 a_i, b_i 。

○ 输入导线

对于属于参与方 P_i 的输入导线， P_i 知道明文导线值 v 。参与方 P_i 将秘密份额 $[v]$ 分发给其他所有参与方。

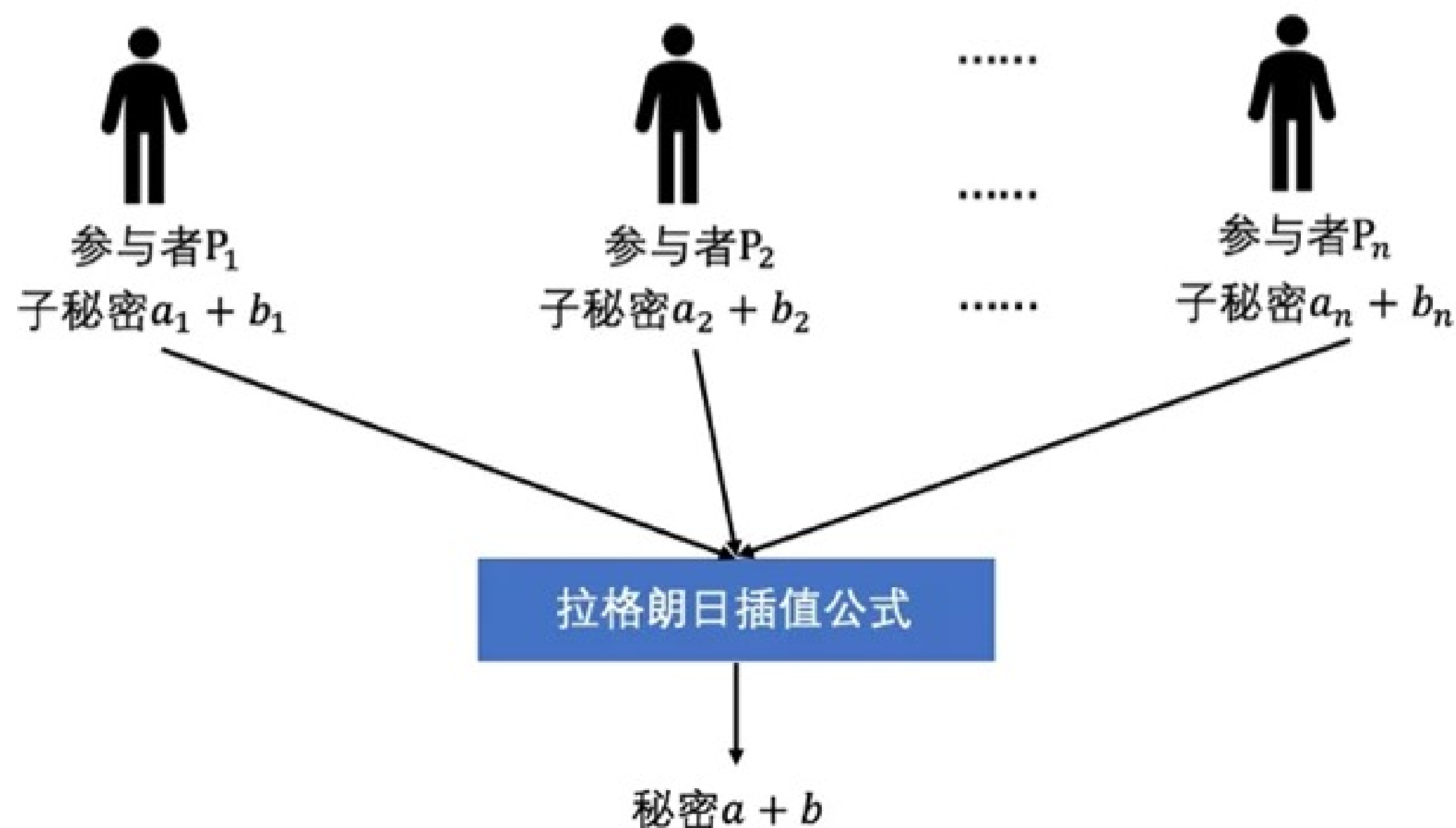
2.具体构造

○ Shamir秘密共享的加法同态性

多方计算 $a+b$: 每个参与者 P_i 独立计算 $c_i=a_i+b_i$, $1 \leq i \leq n$ 。 $c_1, \dots, c_n$ 即为 $a+b$ 通过随机多项式共享后的结果, 通过多项式插值法或者解方程即可恢复秘密 s 。子秘密可以直接相加, 是因为对于:

$$c_i = a_i + b_i = f_a(i) + f_b(i),$$

多项式的次数并没有发生变化, 新的多项式 $f_a(x) + f_b(x)$ 的最高次数依旧为 t , $t+1$ 个参与者共享他们所掌握的 c_i , 即可根据 $t+1$ 个方程解 $t+1$ 个未知数, 解出 $a+b$ 。或者也可直接使用拉格朗日插值法求解出 $a+b$ 。

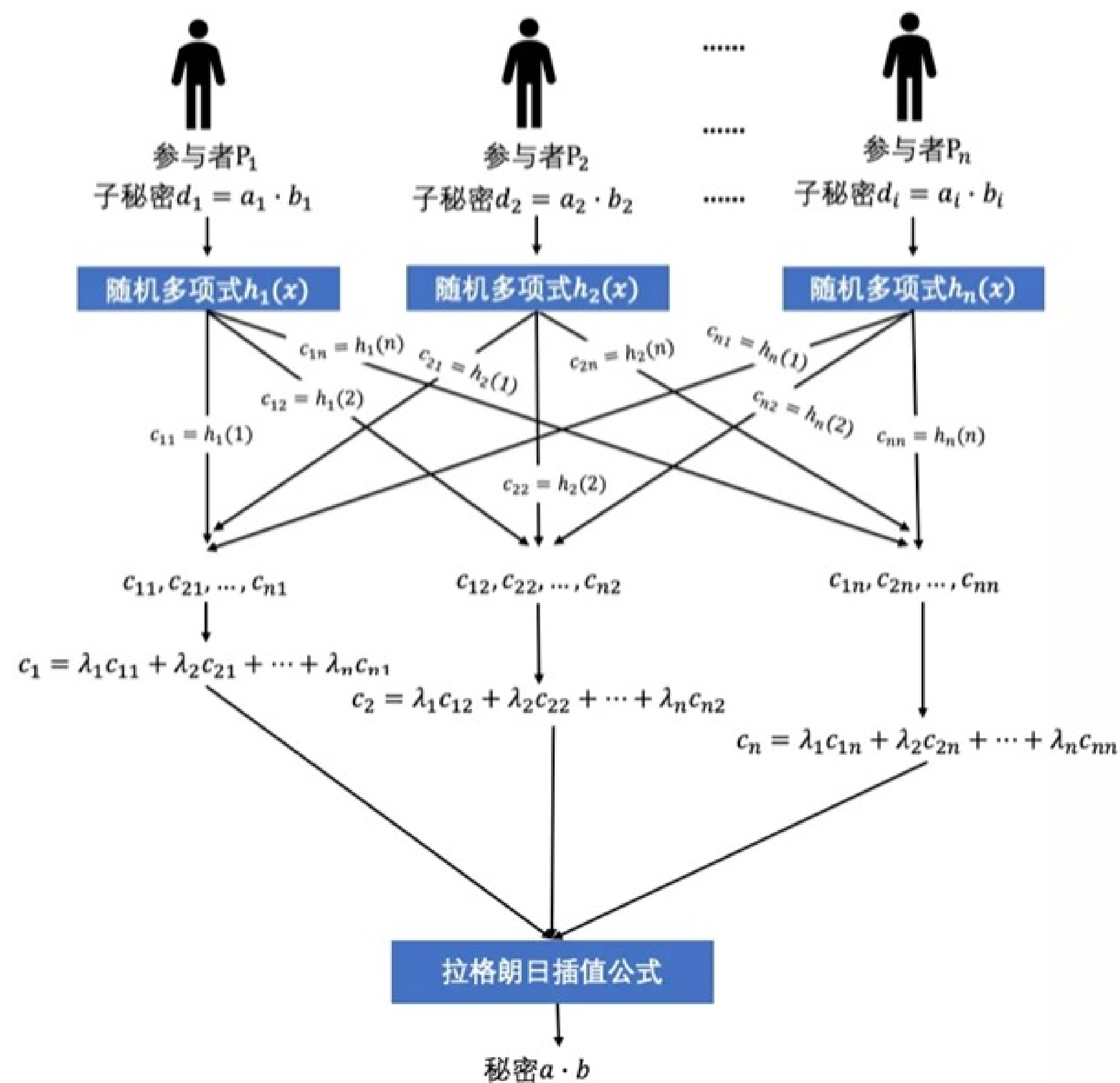


2.具体构造

○ Shamir秘密共享的乘法运算

多方计算 ab : 参与方可以在本地对秘密份额相乘得到 $d_i = a_i \cdot b_i$ ，这使得各个参与方共同持有多项式 $q(x) = p_\alpha(x) \cdot p_\beta(x)$ 上的一个点。然而，得到的多项式阶数最高可达到 $2t$ ，超过了秘密分享的阈值。为了解决秘密分享阈值溢出的问题，各个参与方需要一起完成多项式的降阶步骤。即每个 P_i 独自选取次数为 t 次的随机 n 多项式 $h_i(x)$ ，向各个参与者分配 d_i 。

2.具体构造



对于重组向量存在性的证明过程为： 设

$$f(x) = \sum_{i=0}^{n-1} a_i x^i$$

则： $f(0) = a_0$ 可以表示为：

$$a_0 = (1, 0, 0, \dots, 0)(a_0, a_1, \dots, a_{n-1})^T$$

考虑一个 n 阶矩阵：

$$M = \begin{pmatrix} 1 & 1 & \dots & 1 \\ 1 & 2^1 & \dots & 2^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & n^1 & \dots & n^{n-1} \end{pmatrix}$$

由于矩阵 M 是满秩矩阵，因此存在 $\lambda_1, \dots, \lambda_n \in \mathbb{F}_q$ ，使得：

$$(\lambda_1, \dots, \lambda_n)M = (1, 0, \dots, 0)$$

所以有：

$$\begin{aligned} f(0) &= a_0 = (\lambda_1, \dots, \lambda_n)M(a_0, a_1, \dots, a_{n-1})^T \\ &= (\lambda_1, \dots, \lambda_n)(f(1), f(2), \dots, f(n))^T \\ &= \sum_{i=1}^n \lambda_i f(i) \end{aligned}$$

2.具体构造

○ 加法门

考虑输入导线为 α 和 β 、输出导线为 γ 的加法门。各个参与方共同持有输入导线秘密份额 $[v_\alpha]$ 和 $[v_\beta]$ 。参与方的目标是获得输入导线值 v_α 和 v_β 求和的秘密份额 $[v_\alpha + v_\beta]$ 。假设输入导线值 v_α 和 v_β 所对应的多项式分别为 p_α 和 p_β 。如果每个参与方 P_i 在本地对秘密份额求和，得到 $p_\alpha(i) + p_\beta(i)$ ，则各个参与方将共同持有多项式 $p_\gamma(x) \stackrel{def}{\longrightarrow} p_\alpha(x) + p_\beta(x)$ 上的一个点。由于 p_γ 的阶最高也为 t ，因此各参与方 P_i 所持有的 $p_\alpha(i) + p_\beta(i)$ 构成了 $p_\gamma(0) = p_\alpha(0) + p_\beta(0) = v_\alpha + v_\beta$ 的有效秘密份额。

注意，由于Shamir具有加法同态性，加法门的求值过程不需要参与方之间进行交互。所有计算过程都是在本地完成的。可以利用相同的方法在秘密值上乘以一个公开常数——每个参与方在本地计算秘密份额乘以常数的结果即可。

2.具体构造

○ 乘法门

考虑输入导线为 α 和 β 、输出导线为 γ 的乘法门。各个参与方共同持有输入导线值 v_α 和 v_β 的秘密份额 $[v_\alpha]$ 和 $[v_\beta]$ 。参与方的目标是获得输入导线值 v_α 和 v_β 乘积的秘密份额 $[v_\alpha \cdot v_\beta]$ 。如上所述，参与方可以在本地对秘密份额相乘，这使得各个参与方共同持有多项式 $q(x) = p_\alpha(x) \cdot p_\beta(x)$ 上的一个点。然而，**得到的多项式阶数最高可达到 $2t$ ，超过了秘密分享的阈值。**

为了解决秘密分享阈值溢出的问题，各个参与方需要一起完成多项式的降阶步骤。每个参与方 P_i 持有的秘密份额是 $q(i)$ ，其中 q 是一个阶最高可达到 $2t$ 的多项式。参与方的目标是得到 $q(0)$ 的有效秘密份额，且对应多项式的阶不超过阈值 t 。

2.具体构造

这里要利用的核心结论是，可以用各个参与方秘密份额的线性函数表示 $q(0)$ 。
具体来说，

$$q(0) = \sum_{i=1}^{2t+1} \lambda_i q(i)$$

其中 λ_i 项表示对应的拉格朗日系数。因此，降阶步骤执行过程如下所述：

1. 每个参与方 P_i 生成 $q(i)$ 的 t 阶秘密分享，并将秘密份额 $[q(i)]$ 分发给其他参与方。
为了简化符号表示，我们没有为秘密份额 $[q(i)]$ 所对应的多项式命名。需要记住的是，每个参与方 P_i 选择了最高为 t 阶的多项式，且此多项式的常系数为 $q(i)$ 。
2. 各个参与方在本地计算 $[q(0)] = \sum_{i=1}^{2t+1} \lambda_i [q(i)]$ 。请注意，该表达式仅涉及秘密份额的加法和常数乘法运算。

2.具体构造

由于 $[q(i)]$ 的秘密分享阈值为 t ，因此 $[q(0)]$ 的秘密分享阈值也为 t ，这就满足了固定范式的要求。

请注意，参与方对BGW协议中的乘法门求值时需要进行交互，即各个参与方需要发送秘密份额 $[q(i)]$ 。还需要注意的是，BGW协议要求 $2t+1 \leq n$ ，否则由于 q 的阶可能会达到 $2t$ ， n 个参与方没有足够的信息确定 $q(0)$ 的值，因此当 $2t < n$ 时，BGW协议在 t 个参与方被攻陷的条件下是安全的（即BGW协议的安全性依赖于多数诚实假设）。

○ 输出导线

电路完成求值后，参与方最终会持有输出导线 α 的秘密份额 $[v_\alpha]$ 。每个参与方将秘密份额广播给其他参与方，使得所有参与方都能得到 v_α 。

汇报目录

Beaver三元组

1.直观思想

将MPC协议划分为（参与方输入未知时的）预处理阶段和（参与方选择好输入时的）在线阶段,是一种很受欢迎的MPC协议构造范式。预处理阶段为各个参与方生成一些相互之间具有一定关联性的随机量。参与方于在线阶段可以"消耗"这些随机量。一些主流恶意安全 MPC协议也应用了这一范式。

为了理解如何将协议中的一部分操作转移到预处理阶段, 我们需要回顾一下BGW协议。**BGW** 协议中唯一的实际开销为对每个乘法门求值时的通信开销。然而, 由于这一步骤是在对秘密份额进行操作, 而参与方只能于在线阶段得到秘密份额（也就是说, 秘密份额的取值依赖于电路输入）, 因此将这部分操作转移到预处理阶段好像并不是那么简单。尽管如此, **Beaver**提出了一种非常聪明的方法, 可以将大部分通信量都转移到预处理阶段（**Beaver, 1991**）。

1.直观思想

Beaver三元组（或称乘法三元组）指的是秘密份额三元组 $[a]$ ， $[b]$ ， $[c]$ ，其中 a 和 b 是从某个适当的域中选择出的随机数，而 $c=ab$ 。可以用很多种方法在离线阶段生成Beaver三元组，例如以随机数作为输入直接执行BGW乘法子协议。在线阶段中，每对一个乘法门求值都需要“消耗”一个Beaver三元组。

考虑一个输入导线为 α 和 β 的乘法门。各个参与方持有秘密份额 $[v_\alpha]$ 和 $[v_\beta]$ 。为应用Beaver三元组 $[a]$ ， $[b]$ ， $[c]$ ，计算 $[v_\alpha \cdot v_\beta]$ ，参与方执行下述步骤：

2.具体构造

1. 各个参与方在本地计算 $[v_\alpha - a] = [v_\alpha] - [a]$ ，并打开 $d = v_\alpha - a$

（即所有参与方均向其他参与方告知自己持有的秘密份额 $[d]$ ，就可以得到 d ）。虽然 d 的取值依赖于秘密值 v_α ，但由于秘密值 v_α 被随机值 a 所掩盖，因此打开 d 不会泄漏与 v_α 相关的任何信息。

2. 各个参与方在本地计算 $[v_\beta - b] = [v_\beta] - [b]$ ，并打开 $e = v_\beta - b$ 。

2.具体构造

3. 观察下述等式:

$$\begin{aligned}v_{\alpha}v_{\beta} &= (v_{\alpha} - a + a)(v_{\beta} - b + b) \\&= (d + a)(e + b) \\&= de + db + ae + ab \\&= de + db + ae + c\end{aligned}$$

由于 d 和 e 已被打开, 而各个参与方持有秘密份额 $[a]$, $[b]$, $[c]$, 因此各个参与方在本地即可通过下述公式计算秘密份额 $[v_{\alpha}v_{\beta}]$:

$$[v_{\alpha}v_{\beta}] = de + d[b] + e[a] + [c]$$

2.具体构造

应用这一技术，只需要公开两个参数即可通过本地计算完成乘法门的求值。总的来说，对每个乘法门求值时，每个参与方需要对外广播两个域元素。而在普通BGW协议中，每个参与方需要（通过安全通信信道）发送 n 个域元素。不过，用这种方式比较性能开销实际上忽略了生成Beaver三元组所引入的计算和通信开销。但需要注意，可以通过一些方法批量生成Beaver三元组，使生成每个Beaver三元组的平均开销仅为每个参与方发送常数个域元素（Beerliová-Trubíniová and Hirt, 2008）。

3.使用Beaver三元组的协议抽象

虽然BGW协议（更准确地说是BGW协议的降阶步骤）依赖于Shamir秘密分享方案，但Beaver三元组方法恰当地对BGW协议进行了抽象。实际上，只要“抽象秘密分享方案”的秘密份额 $[v]$ 满足下述性质，就可以使用Beaver三元组方法：

- 加同态性：给定 $[x]$ 、 $[y]$ 和公开值 z ，参与方不需交互即可计算得到 $[x+y]$ 、 $[x+z]$ 以及 $[xz]$ 。
- 可打开性：给定 $[x]$ ，参与方可以选择向所有其他参与方披露 x 。
- 隐私性：攻击者（无论是何种攻击者）都无法从 $[x]$ 中得到与 x 相关的任何信息。
- Beaver三元组：各个参与方可以为每一个乘法门构造满足 $c=ab$ 的随机三元组 $[a]$ ， $[b]$ ， $[c]$ 。
- 随机输入工具：对于属于参与方 P_i 的输入导线，各个参与方可以得到一个随机秘密份额 $[r]$ ，秘密份额 $[r]$ 对于除 P_i 的所有参与方来说都是随机的，只有 P_i 已知 r 。在协议执行过程中，当 P_i 为此条输入导线选择好输入值 x 后， P_i 可以向所有其他参与方公开 $\delta = x - r$ （但这不会泄漏 x 的任何相关信息），参与方可以利用加同态性于本地计算得到 $[x] = [r] + \delta$ 。

3.使用Beaver三元组的协议抽象

只要抽象秘密分享方案满足上述所有性质，Beaver三元组方法就是安全的。进一步，只要抽象秘密分享方案在恶意攻击者的攻击下仍然满足可打开性和隐私性，则Beaver三元组方法也可以抵御恶意攻击者的攻击。如果Beaver三元组方法在恶意攻击者的攻击下是安全的，则恶意攻击者无法伪造出未被打开的秘密值。

目 录

『05』

知识点五： 行约减

『06』

知识点六： FREE XOR

『07』

知识点七： GMW协议

『08』

知识点八： 半树技术

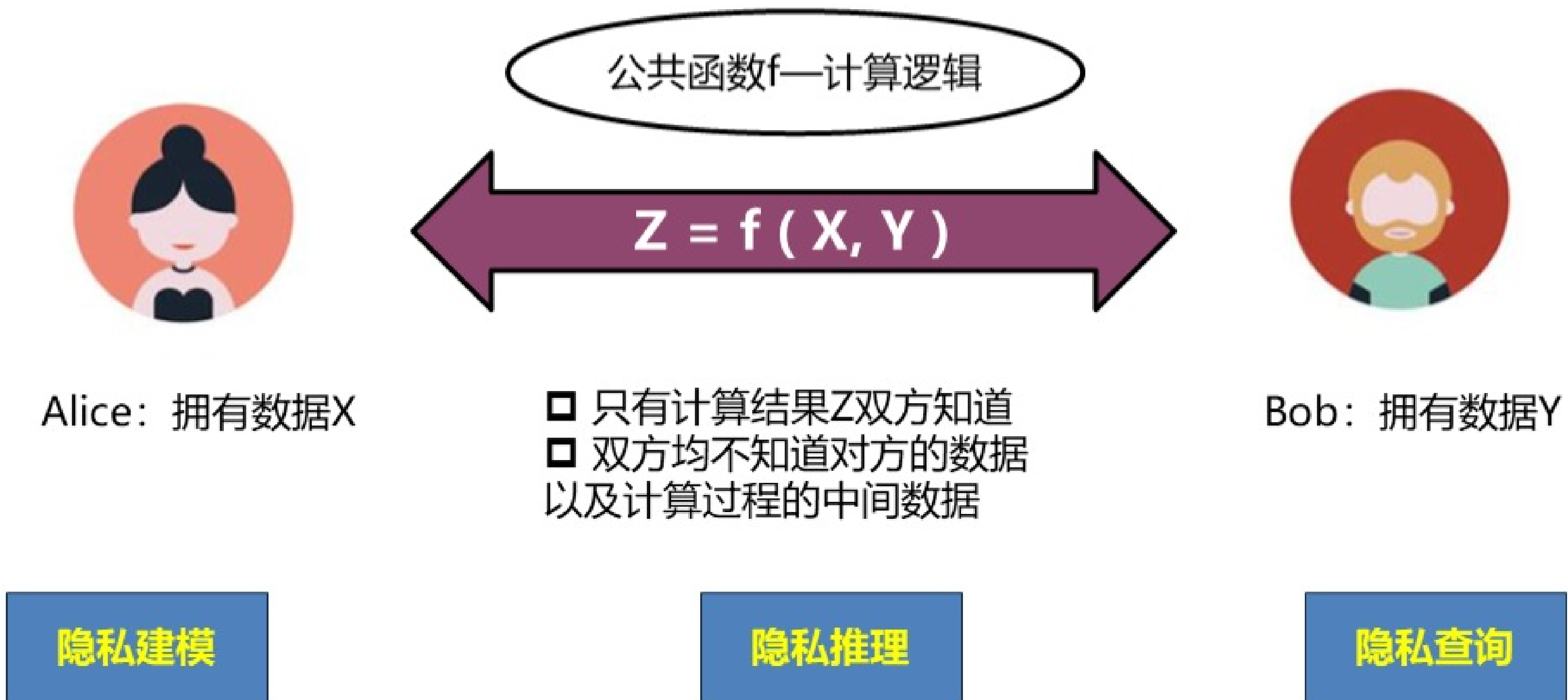
『09』

知识点九： 算术电路协议

研究背景

典型技术：联邦学习（以梯度聚合为主体的联合学习架构）、安全多方计算

安全多方计算：秘密共享、混淆电路、全同态加密 【向混合协议发展】



研究背景

现在安全多方计算非常有用，但性能还存在很多瓶颈

安全两方比较：基于混淆电路的解法

- 一次比较，Alice至少需要向Bob传输326L bits
 - 通信耗时：1000万次比较至少需要100+Gb，即100+秒*
 - 计算耗时：Pipeline技术可以让GC的计算耗时完全被通信耗时掩盖

*按照《隐私计算产品性能测试方法》：
L=32，千兆网。

安全两方乘法

- 基于Beaver triplets的做法
 - 如何生成 $\{x_0, y_0, z_0\}$ 和 $\{x_1, y_1, z_1\}$?
 - 基于同态或OT生成
 - 同态更优[1]
 - 通信代价：2Kbits / triple
 - 计算代价：10us / triple (单核)
- 1000万次乘法需要至少25秒（32核）
 - 绝大部分代价花在生成triple

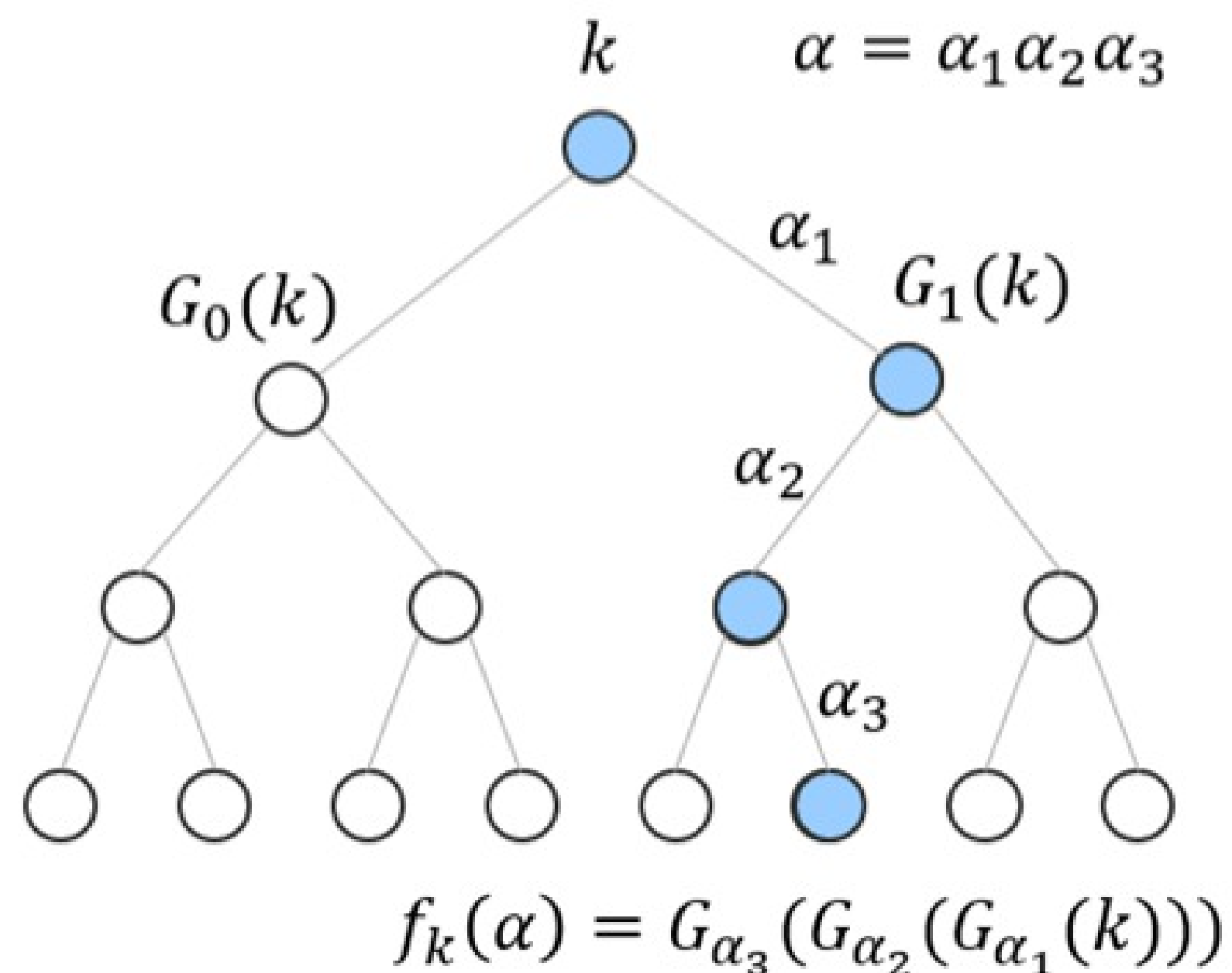
Takeaways

MPC比明文计算慢多少倍	三方计算, 1攻击者		两方计算	
	Honest majority	Dishonest majority	Honest majority	Dishonest majority
Semihonest security	baseline: 2~3个量级	baseline+1个量级		
Malicious security	baseline+2~3倍	baseline+2个量级		

- LAN network
- Training LENET-like models on MNIST
- WAN +1个量级
- 所有数字仅为参考SOTA论文估计，视情况会上下波动（基本都是往更慢波动）

研究动机

GGM树：可以利用一个输出长度是输入两倍的PRG产生一个伪随机函数族。



GGM树

- 关联不经意传输COT
(Correlated OT)
- 子域向量不经意线性估值sVOLE
(subfield Vector Oblivious Linear Evaluation)
- 分布式点函数DPF
(Distributed Point Function)
- 分布式比较函数DCF
(Distributed Comparison Function)

广泛用于具有亚线性通信量的关联随机性生成上

如果能够优化GGM树，则可以使得通用及专用MPC协议的整体性能的提升

关联随机性	具体应用
➤ 关联不经意传输COT ➤ 子域向量不经意线性估值sVOLE	通用MPC [GMW87, ...] 基于VOLE的零知识证明[WYKW21, DIO21, BMRS21] PSI [GPR+21, RS21] , ...
➤ 分布式点函数DPF	基于RAM的MPC[DS17] 两服务器的PIR[GI14,BGI16] OLE扩展[BCG+20] , ...
➤ 分布式比较函数DCF	混合模式的MPC[BGI16, BCG+21] 安全机器学习推理[GKCG22]

难点：GGM缺乏可以提升性能的代数结构

本文贡献

半树：在GGM树构造基础上引入相关性，构造了Correlated GGM (cGGM) 及其随机变种pcGGM，使得整体计算和通信性能平均提升2倍左右，故称之为半树。

设定：UC模型下半诚实安全；恶意安全通过添加一致性检查[\[BCG+21\]](#)很容易达成。

协议	计算量	通信量	轮次
➤ COT	2x	2x	—
➤ sVOLE (版本1)	2x	1~2x	—
➤ sVOLE (版本2)	1.33x	2x	—
➤ DPF	1.33x	3x	2x
➤ DCF	1.6x	2~3x	2x

计算量是以树展开过程的AES调用为准，且不计COT/sVOLE的LPN编码

汇报目录

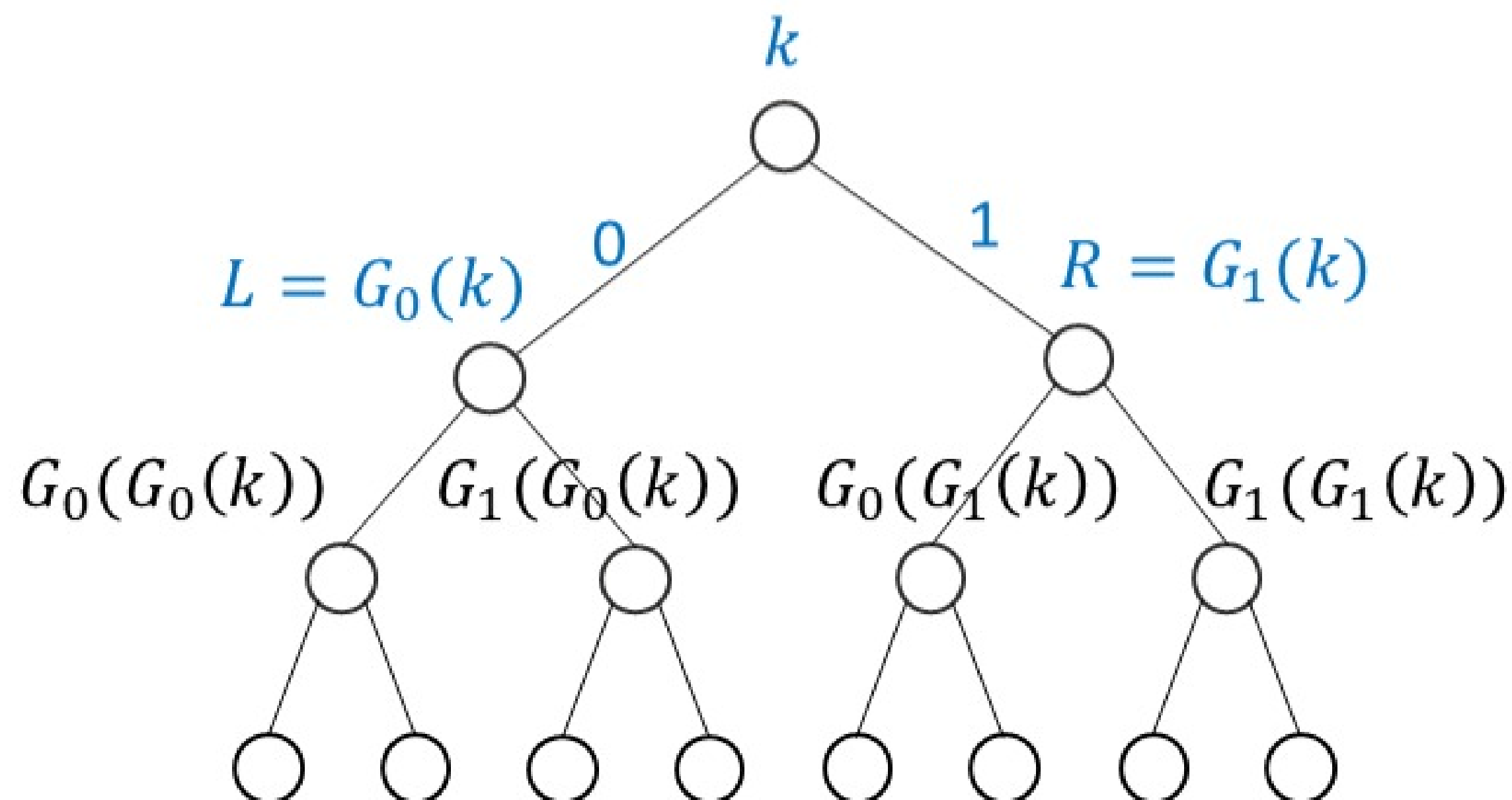
背景知识

1. GGM树

PRG $G: \{0,1\}^\lambda \rightarrow \{0,1\}^{2\lambda}$, G 是一个输出长度是输入两倍的伪随机数生成器

假定根节点的值 $k \in I_\lambda$, I_λ 为长度为 λ 的字符串的集合

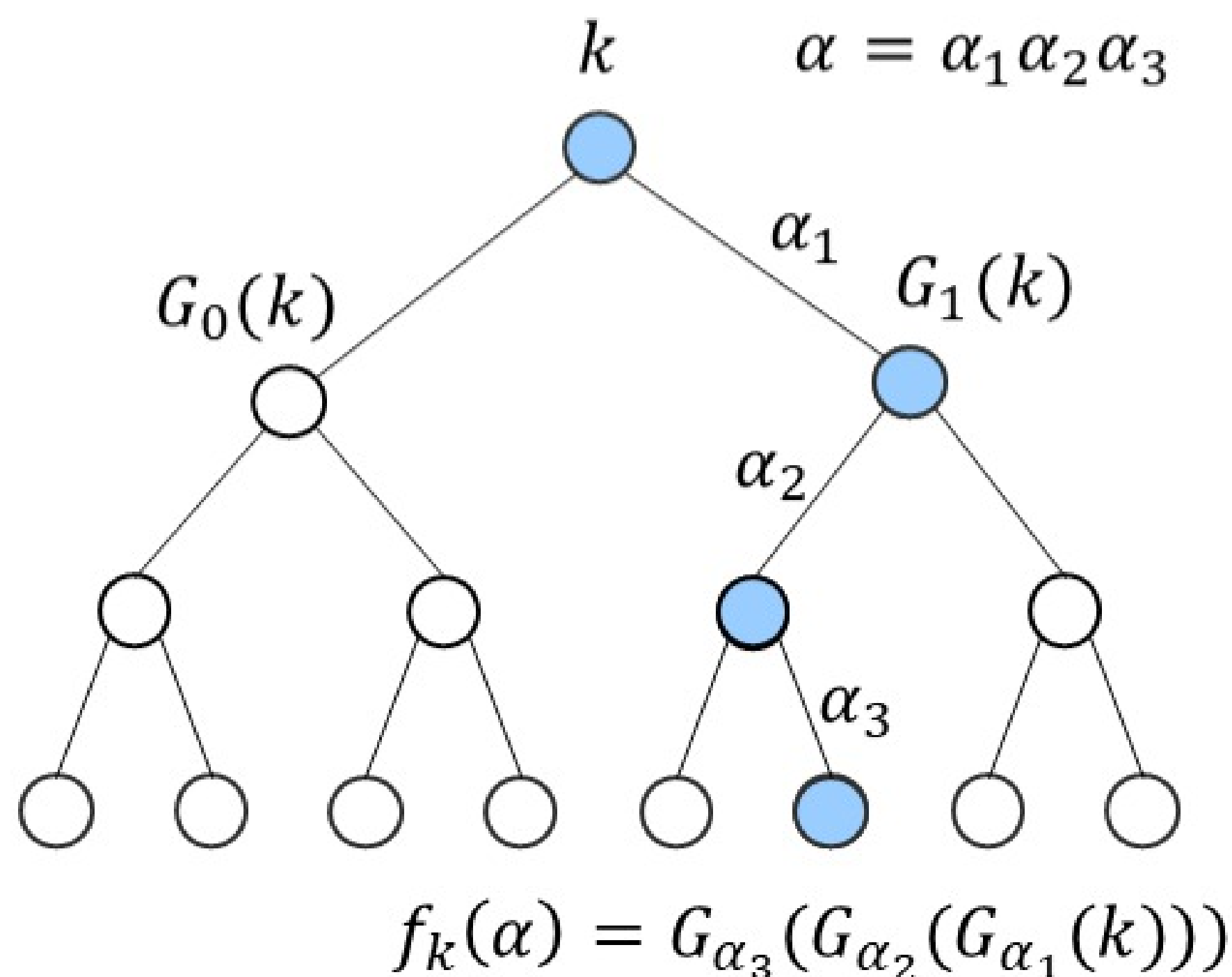
树展开: 以子树根节点值 k 为输入, 通过 $G(k) = G_0(k) || G_1(k)$ 产生左右孩子节点的值



1. GGM树

伪随机函数 $f_k: X \rightarrow Y$ 满足 $f_k(\alpha) = G_{\alpha_\lambda}(\cdots G_{\alpha_2}(G_{\alpha_1}(k))\cdots)$, 其中 $\alpha = \alpha_1 \cdots \alpha_{|x|}$

对于任意输入 α , 根据其二进制位逐位进行树展开, 即可得到伪随机函数的输出



单向性

2. 茫然传输

茫然传输 (Oblivious Transfer, OT) 是安全多方计算协议的重要构造模块。

2选1-OT的标准定义涉及两个参与方：持有两个秘密值 x_0, x_1 的发送方，持有一个选择比特 $b \in \{0,1\}$ 的接收方。OT允许接收方得到 x_b ，但其无法得到与“另一个”秘密值 x_{1-b} 相关的任何信息。与此同时，发送方无法得到任何信息。

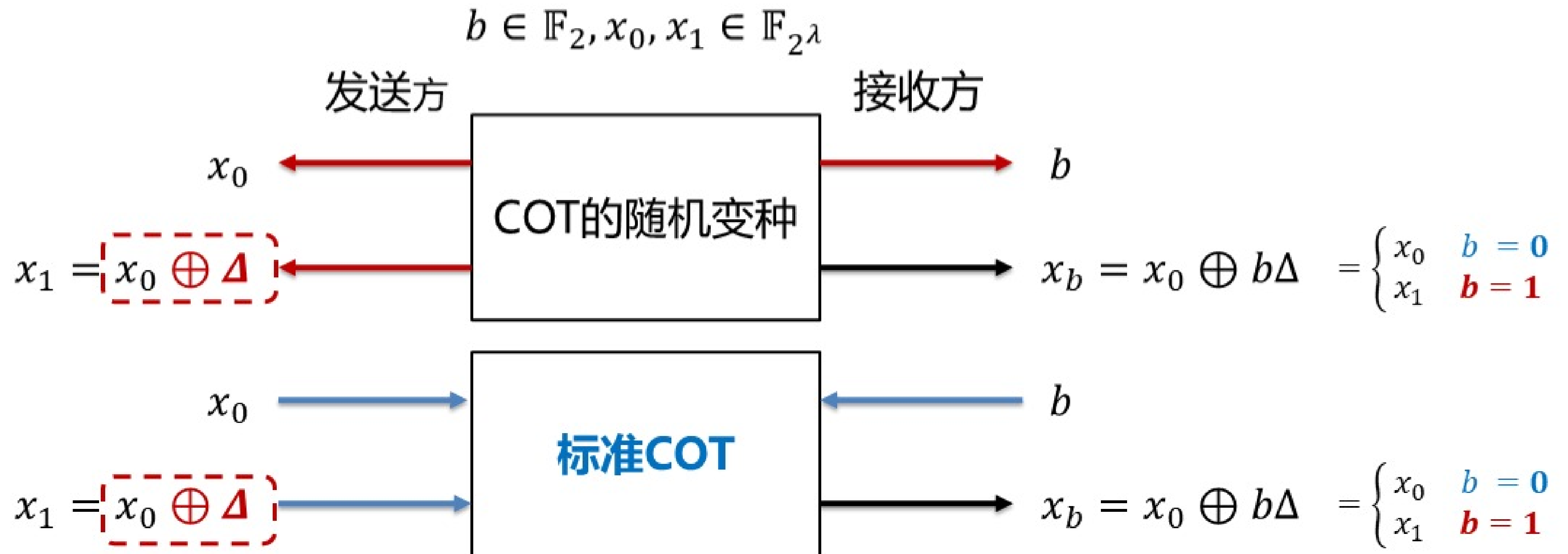


2. 茫然传输

关联COT：发送方输入两个具有相关性的消息 x_0, x_1 ，接收方输入选择比特 b ，收到 x_b 。

COT的随机变种：发送方获得具有相关性的 x_0, x_1 ，接收方收到随机的 $b \in \{0,1\}$ 及 x_b 。

COT的随机变种 $\Rightarrow$ ROT $\Rightarrow$ OT。



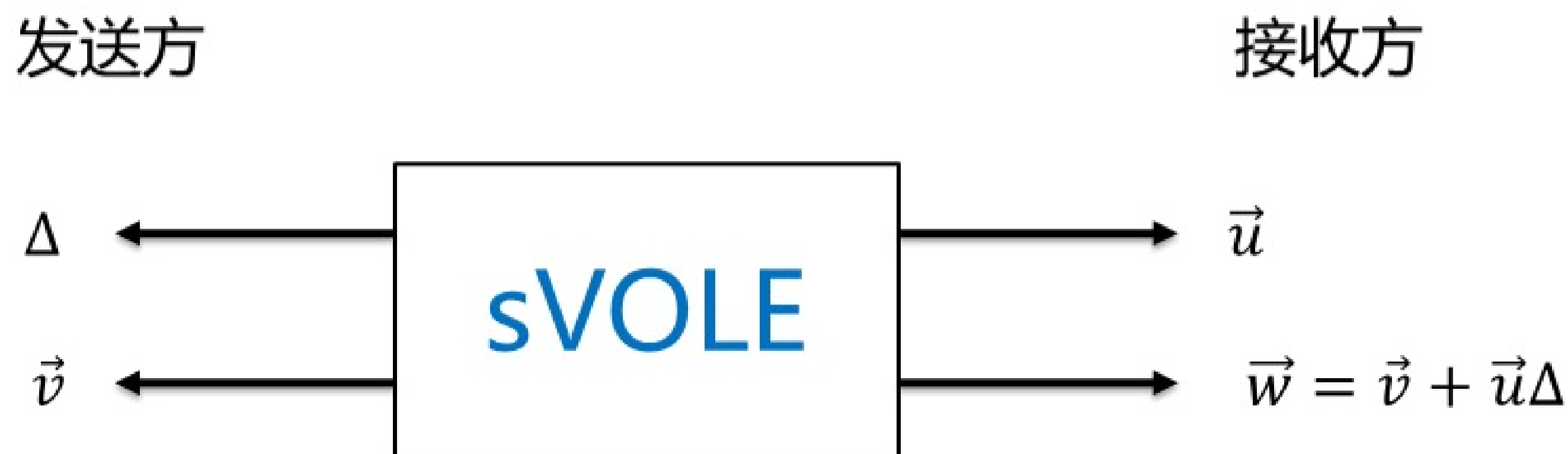
3. 子域向量不经意线性估值

sVOLE参数：域 $\mathbb{F}$ 和它的扩张域 $\mathbb{K}$ 。

sVOLE输出：给发送方分配 $(\Delta, \vec{v}) \in \mathbb{K} \times \mathbb{K}^m$ ，给接收方分配 $(\vec{u}, \vec{w}) \in \mathbb{F}^m \times \mathbb{K}^m$ 。

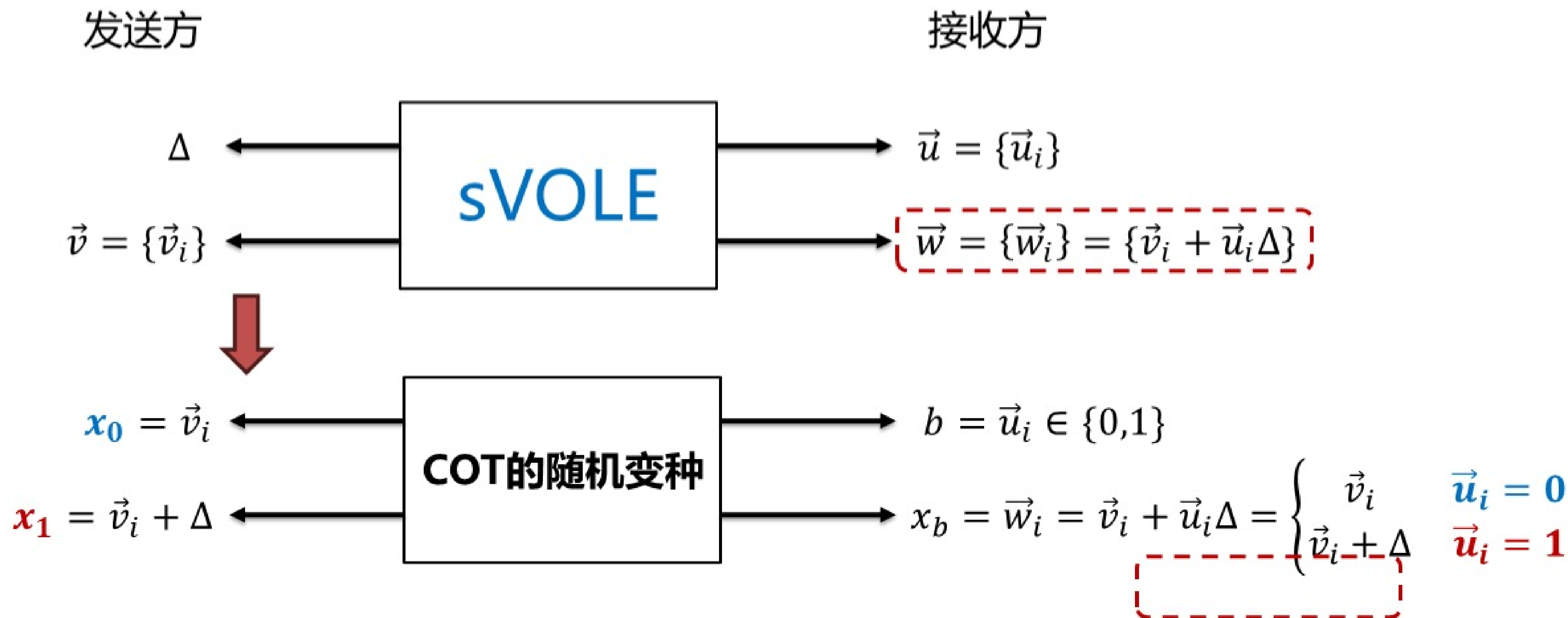
sVOLE关联性： $\vec{w} = \vec{v} + \vec{u}\Delta$ 。

VOLE是sVOLE的特殊情况：当 $\mathbb{F} = \mathbb{K}$ 时，sVOLE可以转换为VOLE。



3. 子域向量不经意线性估值

COT是sVOLE的特殊情况：当 $\mathbb{F} = \mathbb{F}_2, \mathbb{K} = \mathbb{F}_{2^\lambda}$ 时，sVOLE可以转换为COT。

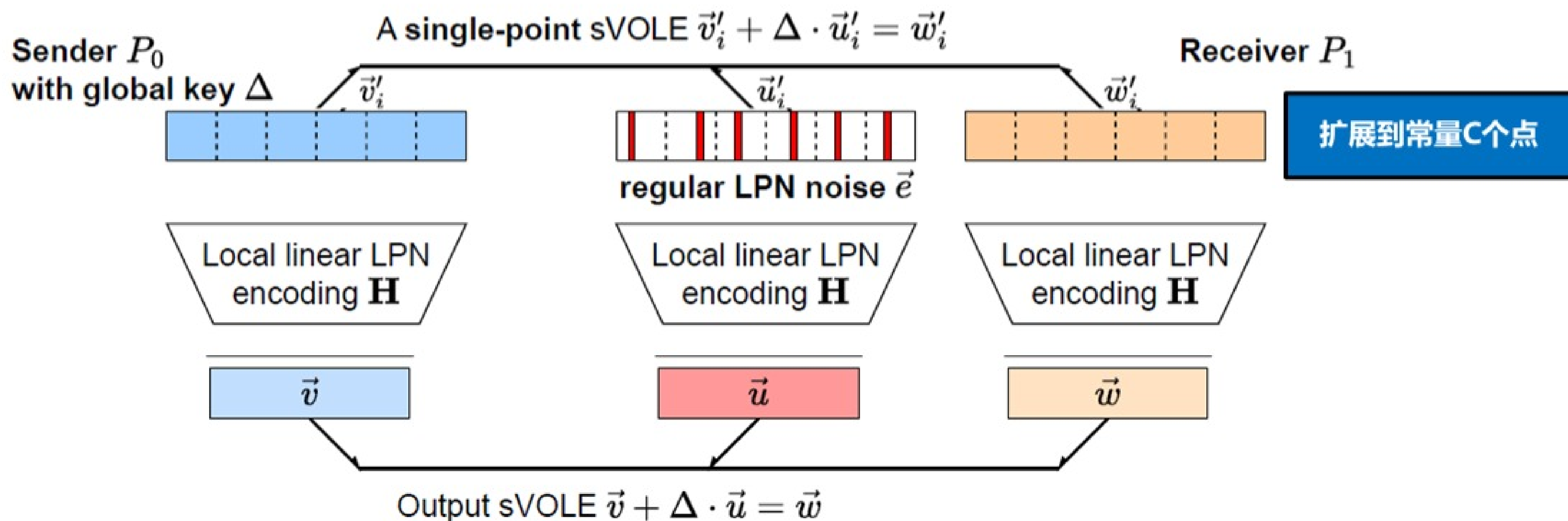


4. 构造COT/sVOLE

构造COT/sVOLE: Single-point sVOLE + LPN编码 = sVOLE

单点COT/sVOLE: $\vec{u}$ 的汉明重量为1, 即 $\vec{u}$ 中非零元素个数为1

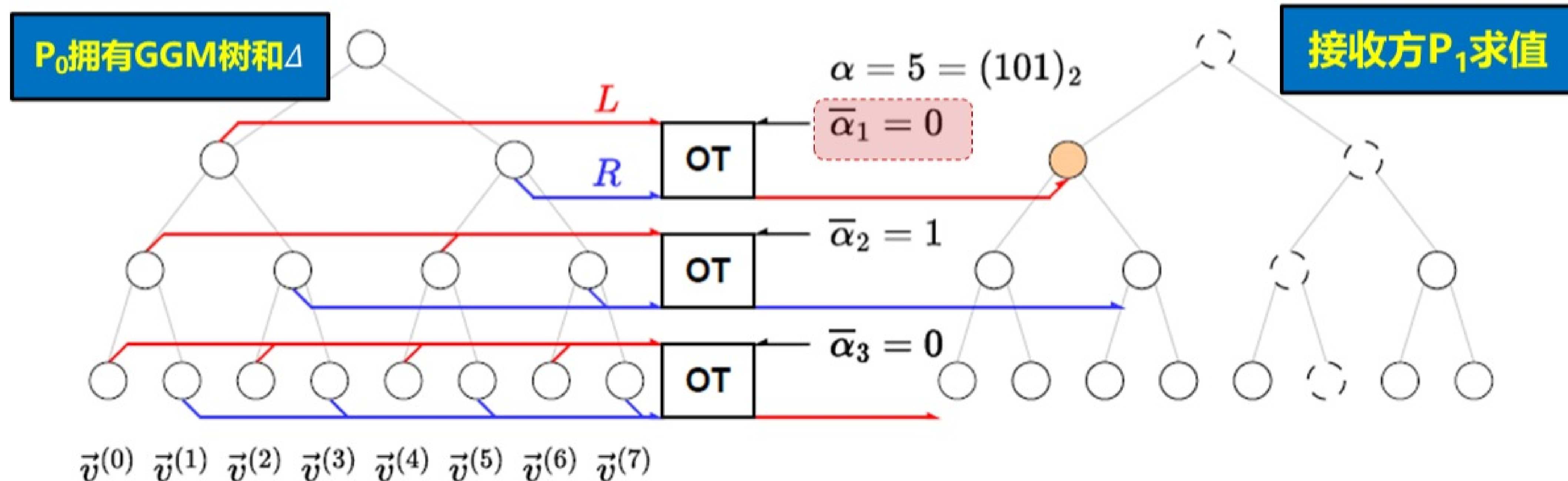
→ COT非零元素值为1, sVOLE非零元素为任意值



本地线性LPN编码将C个 $\vec{u}'$ 压缩为一个 $\vec{u}$ ，使其几乎均匀随机

5. 构造单点COT/sVOLE

假定单点COT/sVOLE中，向量 $\vec{u}$ 不为零的元素位置为 $\alpha=5=(101)_2$

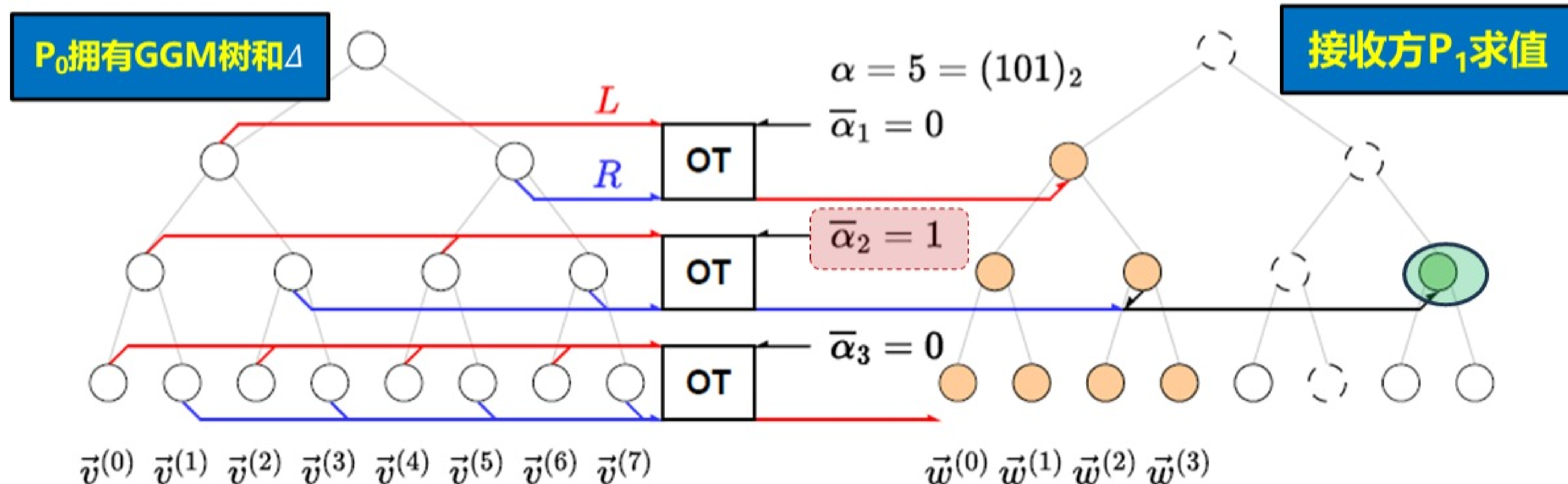


□ 由于 α 的每一位对应一层，选取对应层中相反的兄弟节点，执行OT

□ 考虑安全性，每次获得对应层所有左节点或者右节点的异或值

5. 构造单点COT/sVOLE

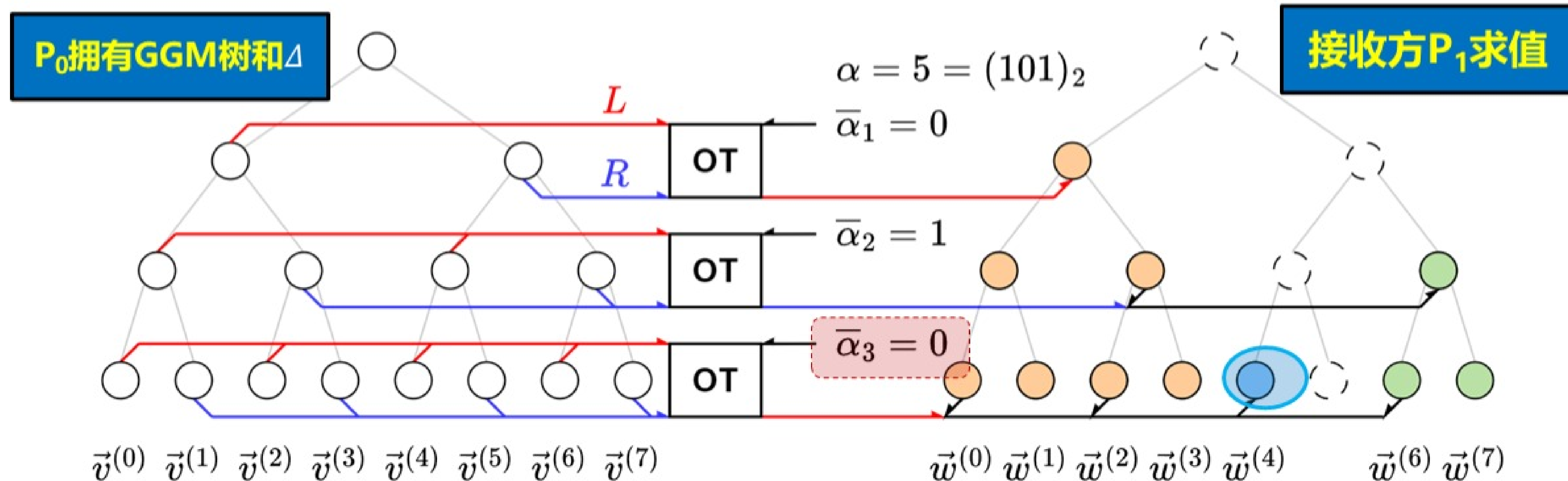
假定单点COT/sVOLE中，向量 $\vec{u}$ 不为零的元素位置为 $\alpha=5=(101)_2$



□ 第二层，得到 P_0 所有右节点的异或值之后， P_1 本地执行异或运算即可得到结果

5. 构造单点COT/sVOLE

假定单点COT/sVOLE中，向量 $\vec{u}$ 不为零的元素位置为 $\alpha=5=(101)_2$

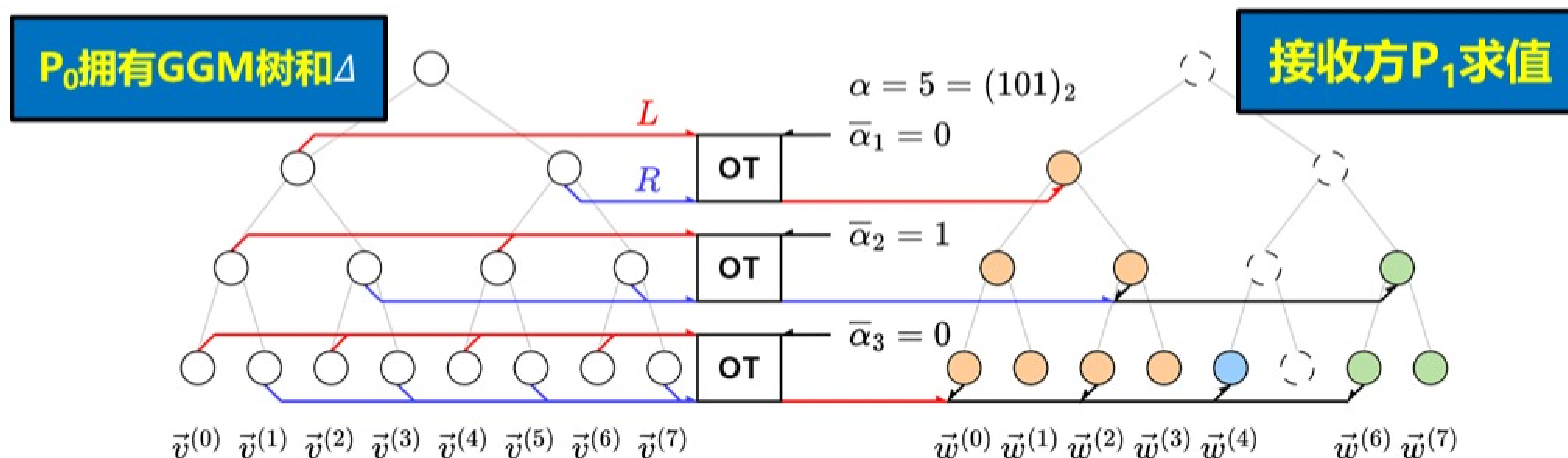


□ 最后一层，得到 P_0 所有左节点的异或值之后， P_1 本地执行异或运算即可得到结果

□ 通过三次OT， P_1 得到除了 $\vec{w}^{(5)}$ 以外的所有 $\vec{w}^{(i)}$ ，并且 $\vec{w}^{(i)} = \vec{v}^{(i)}, i \neq 5$ ， $\vec{w}^{(5)}$ ？

5. 构造单点COT/sVOLE

假定单点COT/sVOLE中，向量 $\vec{u}$ 不为零的元素位置为 $\alpha=5=(101)_2$



$$\psi := \Delta \oplus (\oplus_{i \in [0,8)} \vec{v}^{(i)}) \longrightarrow \vec{w}^{(5)} = \psi \oplus (\oplus_{i \neq 5} \vec{w}^{(i)}) = \Delta \oplus \vec{v}^{(5)}$$

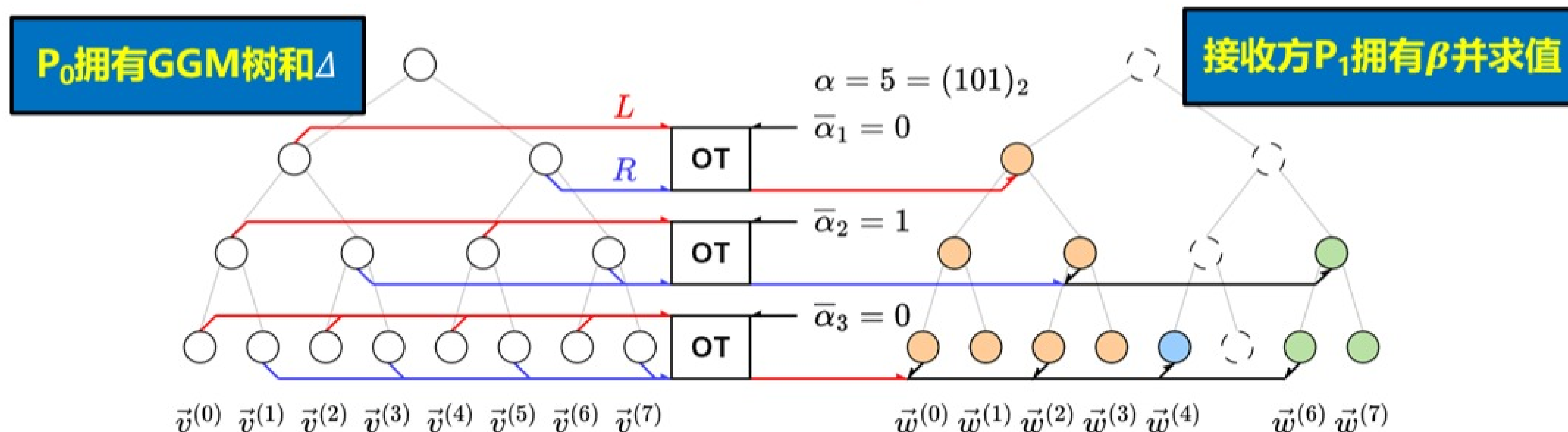


单点COT

- 当 $i \neq \alpha$ 时, $\vec{u}^{(i)} = 0$, $\vec{w}^{(i)} = \vec{v}^{(i)} \oplus \vec{u}^{(i)} \Delta = \vec{v}^{(i)}$
- 当 $i = \alpha$ 时, $\vec{u}^{(i)} = 1$, $\vec{w}^{(i)} = \vec{v}^{(i)} \oplus \vec{u}^{(i)} \Delta = \vec{v}^{(i)} \oplus \Delta$

5. 构造单点COT/sVOLE

在单点COT中, $\vec{u}^{(\alpha)} = 1$, 在单点sVOLE中, P_1 拥有 $\vec{u}^{(\alpha)} = \beta$, 因此两者构造有不同



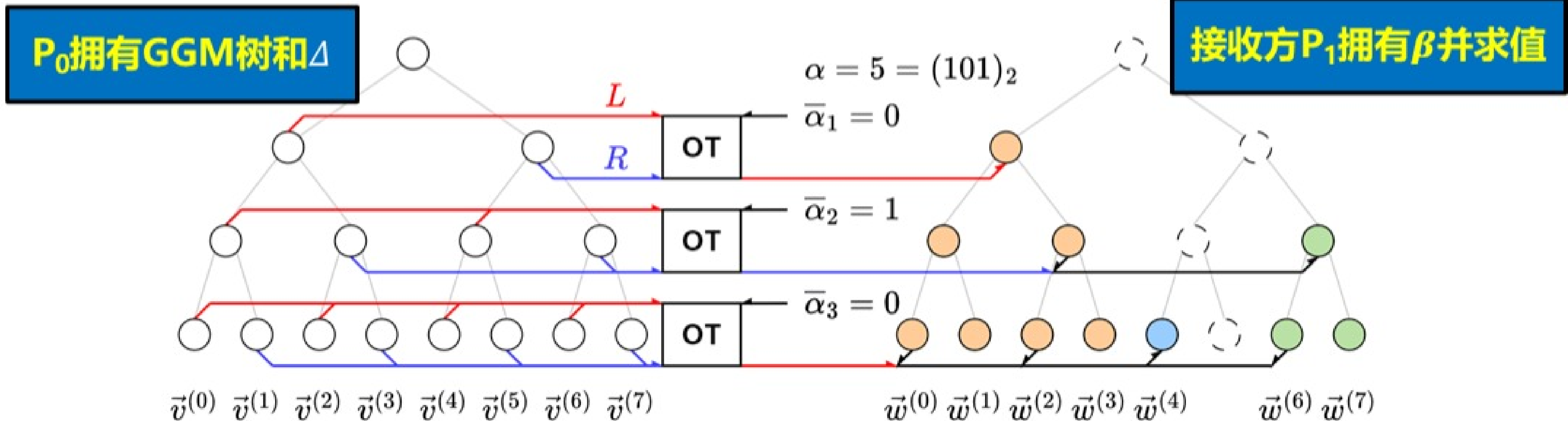
$$\psi := \Delta \oplus (\oplus_{i \in [0,8)} \vec{v}^{(i)}) \text{ ? } \beta \text{ ?} \longrightarrow \vec{w}^{(5)} = \psi \oplus (\oplus_{i \neq 5} \vec{w}^{(i)}) \neq \beta \Delta \oplus \vec{v}^{(5)}$$

?

因为P₀有 Δ , 但是P₁有 β , 单方是不可能构造出 $\beta\Delta$
必须依赖两方计算

5. 构造单点COT/sVOLE

在单点COT中, $\vec{u}^{(\alpha)} = 1$, 在单点sVOLE中, P_1 拥有 $\vec{u}^{(\alpha)} = \beta$, 因此两者构造有不同



$$\begin{array}{c}
 \Delta, K_\beta \xleftarrow{\quad} \boxed{\begin{array}{c} \text{一个预处理的随机sVOLE元组} \\ M_\beta = K_\beta + \beta \cdot \Delta \end{array}} \xrightarrow{\quad} \beta, M_\beta \\
 \psi := -K_\beta + (\sum_{i \in [0,8)} \vec{v}^{(i)}) \xrightarrow{\quad} \vec{w}^{(5)} := M_\beta + \psi - (\sum_{i \neq 5} \vec{w}^{(i)})
 \end{array}$$

- 单点sVOLE**
- 当 $i \neq \alpha$ 时, $\vec{u}^{(i)} = 0$, $\vec{w}^{(i)} = \vec{v}^{(i)} + \vec{u}^{(i)} \Delta = \vec{v}^{(i)}$
 - 当 $i = \alpha$ 时, $\vec{u}^{(i)} = \beta$, $\vec{w}^{(i)} = \vec{v}^{(i)} + \vec{u}^{(i)} \Delta = \vec{v}^{(i)} + \beta \Delta$

汇报目录

半树设计

1. 关联GGM树 (Correlated GGM, cGGM)

全局偏移量

$$\Delta \stackrel{\Sigma}{\leftarrow}$$

$$\Delta \stackrel{\Sigma}{\leftarrow}$$

$$\Delta \stackrel{\Sigma}{\leftarrow}$$

$$k \leftarrow \mathbb{K}$$

"根"

$$\Delta \oplus k$$

$$H(k) \quad k \oplus H(k)$$

$$H(\Delta \oplus k) \quad \Delta \oplus k \oplus H(\Delta \oplus k)$$

哈希函数: $H(x) := \pi(\sigma(x)) + \sigma(x)$, 其中 π 是一个随机置换, σ 是可计算的线性正交。[GKWY20]

➤ $\pi : \mathbb{K} \rightarrow \mathbb{K}$ 是一个随机置换

➤ $\sigma : \mathbb{K} \rightarrow \mathbb{K}$ 是一个有效可计算线性正射

- σ 和 $\sigma : x \rightarrow \sigma(x) - x$ 都是置换, $\sigma(x + y) = \sigma(x) + \sigma(y)$

- [GKWY20]给出了一个候选的构造方式

1. 关联GGM树 (Correlated GGM, cGGM)

全局偏移量

$$\Delta \stackrel{\Sigma}{\leftarrow}$$

$$\Delta \stackrel{\Sigma}{\leftarrow}$$

$$\Delta \stackrel{\Sigma}{\leftarrow}$$

$$k \leftarrow \mathbb{K}$$

"根"

$$\Delta \oplus k$$

$$H(k) \quad k \oplus H(k)$$

$$H(\Delta \oplus k) \quad \Delta \oplus k \oplus H(\Delta \oplus k)$$

□ 根节点: $k \oplus (\Delta \oplus k) = \Delta$

□ 树展开: 左孩子=H(父亲节点) 右孩子=父亲节点 $\oplus$ 左孩子

→ 第一、计算减半! 第二、左孩子 $\oplus$ 右孩子=父亲节点

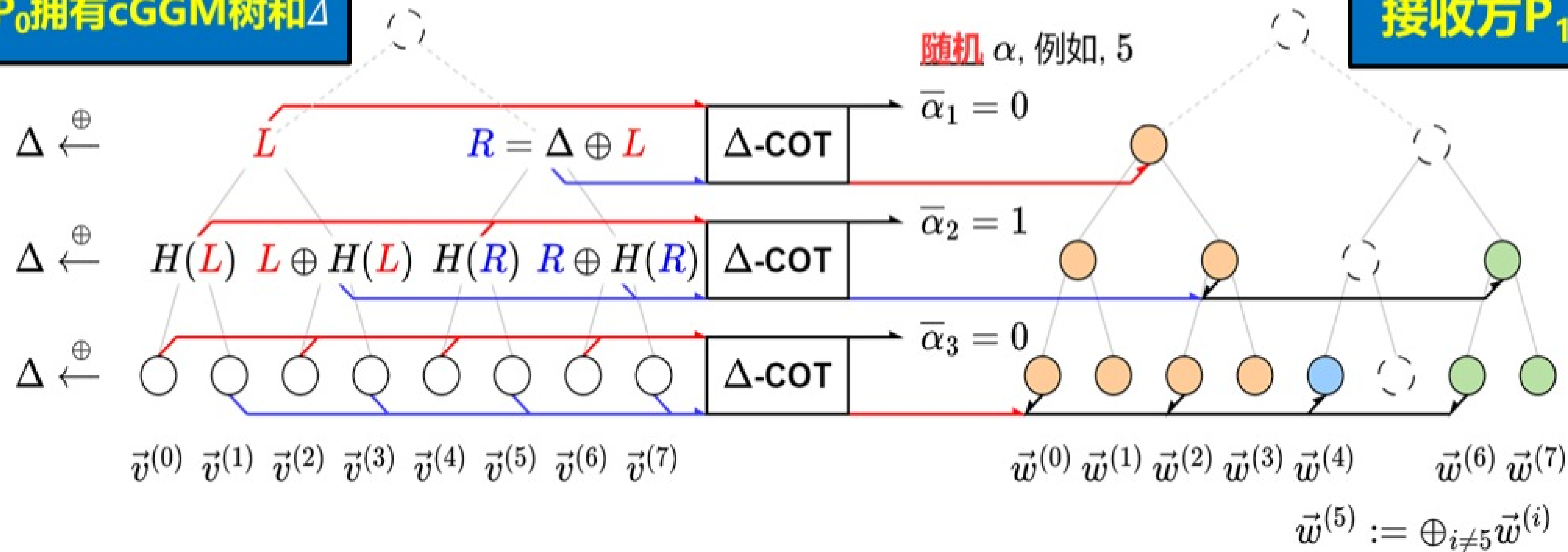
□ 相同层、能展开左右子树的所有节点异或值都为全局偏移量

2. 用cGGM构造单点COT

单点COT: $\alpha=5=(101)_2$

P_0 拥有cGGM树和 Δ

接收方 P_1 求值



□ 用cGGM树代替GGM树，每一层节点值异或为 Δ

□ 树展开：左孩子为 $H(L)$ ，右孩子为 $L \oplus H(L)$

→ 计算减半

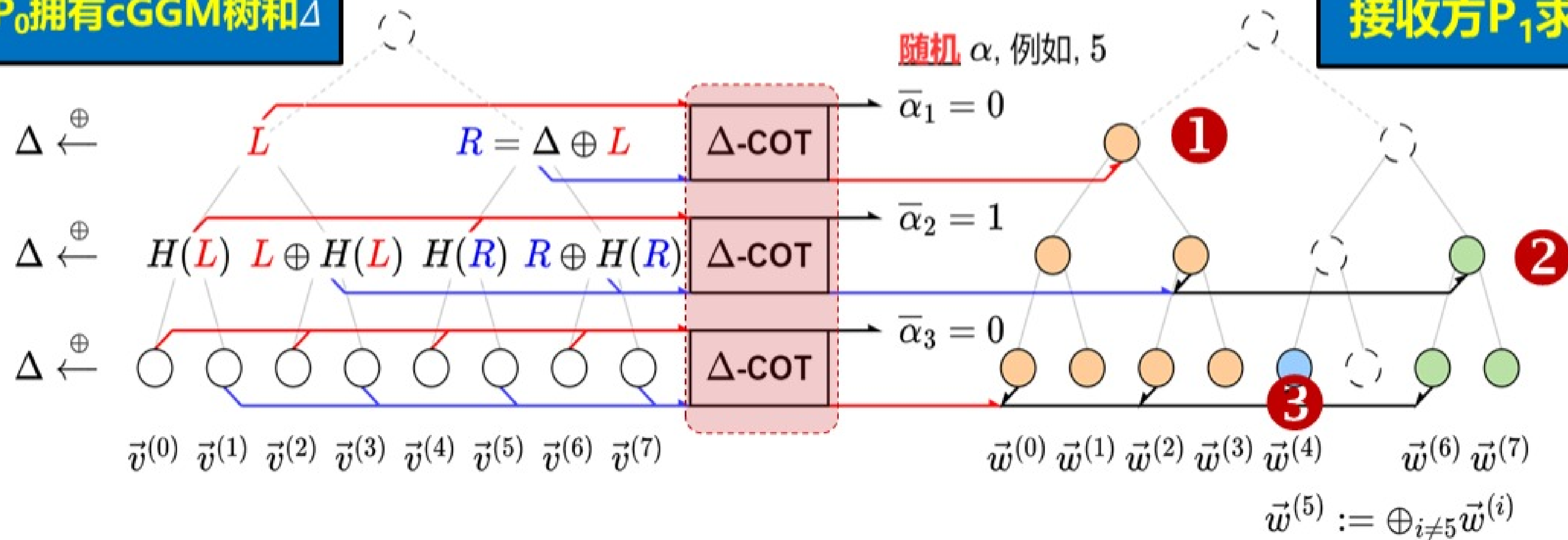
□ 用 L_i , R_i 表示每层左右孩子异或值, $L_i \oplus R_i = \Delta$

2. 用cGGM构造单点COT

单点COT: $\alpha=5=(101)_2$

P_0 拥有cGGM树和 Δ

接收方 P_1 求值



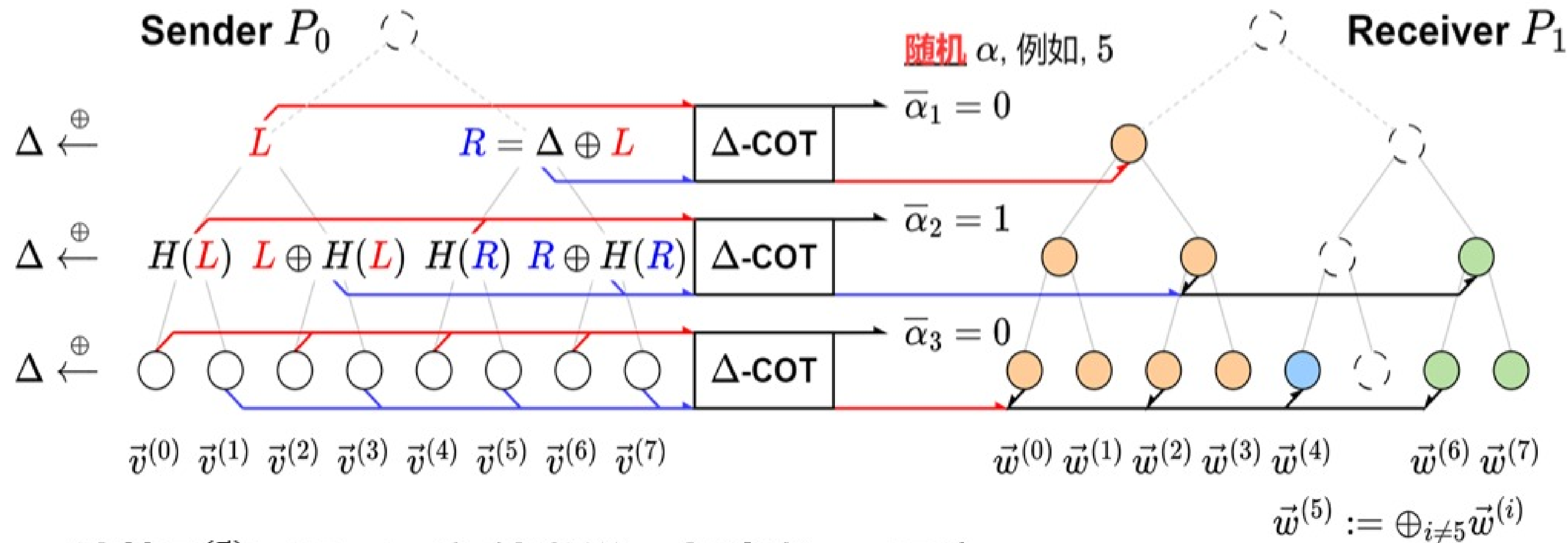
P_1 求值过程时, 对每一层执行 $\Delta - COT$ (通信减半):

P_0 输入 L_i , P_1 得到 $\bar{\alpha}_i$ 对应的 L_i 或 R_i , 然后异或求值可以得到节点的值

传统OT需要发送两个 λ 比特长数据, $\Delta - COT$ 只需要发送一个 λ 比特的数据

2. 用cGGM构造单点COT

单点COT: $\alpha=5=(101)_2$



- 计算 $\vec{w}^{(5)}$ 不需要 P_0 额外发送一条消息 ψ , 因为:
 - cGGM要求每层节点和为 Δ , $\oplus_{i \in [0,8)} \vec{v}^{(i)} = \Delta \Rightarrow \Delta \oplus \vec{v}^{(5)} = \oplus_{i \neq 5} \vec{v}^{(i)}$
 - 所以, 直接本地计算: $\vec{w}^{(5)} = \Delta \oplus \vec{v}^{(5)} = \oplus_{i \neq 5} \vec{w}^{(i)}$ (符合COT的定义)
- (因此cGGM树同层所有节点异或为 Δ 的特性, 省掉了本次消息发送)

谢 谢

